

My LeetCode Submissions - @shauryad15

[Download PDF](#)[Follow @TheShubham99 on GitHub](#)[Star on GitHub](#)[View Source Code](#)

15 3Sum (link)

Description

Given an integer array `nums`, return all the triplets `[nums[i], nums[j], nums[k]]` such that $i \neq j$, $i \neq k$, and $j \neq k$, and $nums[i] + nums[j] + nums[k] == 0$.

Notice that the solution set must not contain duplicate triplets.

Example 1:

Input: `nums = [-1,0,1,2,-1,-4]`

Output: `[[-1,-1,2],[-1,0,1]]`

Explanation:

`nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.`

`nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.`

`nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.`

The distinct triplets are `[-1,0,1]` and `[-1,-1,2]`.

Notice that the order of the output and the order of the triplets does not matter

Example 2:

Input: `nums = [0,1,1]`

Output: `[]`

Explanation: The only possible triplet does not sum up to 0.

Example 3:

Input: `nums = [0,0,0]`

Output: `[[0,0,0]]`

Explanation: The only possible triplet sums up to 0.

Constraints:

- $3 \leq \text{nums.length} \leq 3000$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def threeSum(self, nums: List[int]) -> List[List[int]]:
        nums.sort()
        result = []
        for i in range(len(nums)):
            if i > 0 and nums[i] == nums[i-1]:
                continue
            left = i + 1
            right = len(nums) - 1
            while left < right:
                if nums[i] + nums[left] + nums[right] == 0:
                    result.append([nums[i], nums[left], nums[right]])
                    left += 1
                    right -= 1
                    #skip duplicates
                    while left < right and nums[left] == nums[left-1]:
                        left += 1
                    while left < right and nums[right] == nums[right+1]:
                        right -= 1
                elif nums[i] + nums[left] + nums[right] < 0:
                    left += 1
                else:
                    right -= 1

        return result
```

[2116 Count Number of Pairs With Absolute Difference K](#)

Description

Given an integer array `nums` and an integer `k`, return *the number of pairs* (i, j) *where* $i < j$ *such that* $|\text{nums}[i] - \text{nums}[j]| == k$.

The value of $|x|$ is defined as:

- x if $x \geq 0$.
- $-x$ if $x < 0$.

Example 1:

Input: `nums = [1,2,2,1], k = 1`
Output: 4
Explanation: The pairs with an absolute difference of 1 are:
- `[1,2,2,1]`
- `[1,2,2,1]`
- `[1,2,2,1]`
- `[1,2,2,1]`

Example 2:

Input: `nums = [1,3], k = 3`
Output: 0
Explanation: There are no pairs with an absolute difference of 3.

Example 3:

Input: `nums = [3,2,1,5,4], k = 2`
Output: 3
Explanation: The pairs with an absolute difference of 2 are:
- `[3,2,1,5,4]`
- `[3,2,1,5,4]`
- `[3,2,1,5,4]`

Constraints:

- $1 \leq \text{nums.length} \leq 200$
- $1 \leq \text{nums}[i] \leq 100$
- $1 \leq k \leq 99$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def countKDifference(self, nums: List[int], k: int) -> int:
        freq = {}
        count = 0

        for num in nums:
            if num - k in freq:
                count += freq[num-k]
            if num + k in freq:
                count += freq[num + k]
            freq[num] = freq.get(num, 0) + 1
        return count
```

[56 Merge Intervals \(link\)](#)

Description

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals, and return *an array of the non-overlapping intervals that cover all the intervals in the input*.

Example 1:

Input: `intervals = [[1,3],[2,6],[8,10],[15,18]]`

Output: `[[1,6],[8,10],[15,18]]`

Explanation: Since intervals `[1,3]` and `[2,6]` overlap, merge them into `[1,6]`.

Example 2:

Input: `intervals = [[1,4],[4,5]]`

Output: `[[1,5]]`

Explanation: Intervals `[1,4]` and `[4,5]` are considered overlapping.

Constraints:

- $1 \leq \text{intervals.length} \leq 10^4$
- $\text{intervals}[i].\text{length} == 2$
- $0 \leq \text{start}_i \leq \text{end}_i \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def merge(self, intervals: List[List[int]]) -> List[List[int]]:
        intervals.sort(key = lambda x:x[0])
        merged = [intervals[0]]

        for i in range(1, len(intervals)):
            if merged[-1][1] >= intervals[i][0]:
                merged[-1][1] = max(merged[-1][1], intervals[i][1])
            else:
                merged.append(intervals[i])

        return merged
```

[88 Merge Sorted Array \(link\)](#)

Description

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be *stored inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to `0` and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`

Output: `[1,2,2,3,5,6]`

Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.

The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from

Example 2:

Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`

Output: `[1]`

Explanation: The arrays we are merging are `[1]` and `[]`.

The result of the merge is `[1]`.

Example 3:

Input: `nums1 = [0]`, `m = 0`, `nums2 = [1]`, `n = 1`

Output: `[1]`

Explanation: The arrays we are merging are `[]` and `[1]`.

The result of the merge is `[1]`.

Note that because `m = 0`, there are no elements in `nums1`. The `0` is only there to e

Constraints:

- `nums1.length == m + n`
- `nums2.length == n`
- `0 <= m, n <= 200`
- `1 <= m + n <= 200`
- `-109 <= nums1[i], nums2[j] <= 109`

Follow up: Can you come up with an algorithm that runs in $O(m + n)$ time?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def merge(self, nums1: List[int], m: int, nums2: List[int], n: int) -> None:
        """
        Do not return anything, modify nums1 in-place instead.
        """
        index = m + n - 1
        m -= 1
        n -= 1
        while m >= 0 or n >= 0:
            if m >= 0 and n >= 0:
                if nums1[m] >= nums2[n]:
                    nums1[index] = nums1[m]
                    m -= 1
                    index -= 1
                else:
                    nums1[index] = nums2[n]
                    n -= 1
                    index -= 1
            elif m >= 0:
                nums1[index] = nums1[m]
                m -= 1
                index -= 1
            else:
                nums1[index] = nums2[n]
                n -= 1
                index -= 1

        return nums1
```

[152 Maximum Product Subarray \(link\)](#)

Description

Given an integer array `nums`, find a subarray that has the largest product, and return *the product*.

The test cases are generated so that the answer will fit in a **32-bit** integer.

Example 1:

Input: `nums = [2,3,-2,4]`
Output: 6
Explanation: `[2,3]` has the largest product 6.

Example 2:

Input: `nums = [-2,0,-1]`
Output: 0
Explanation: The result cannot be 2, because `[-2,-1]` is not a subarray.

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-10 \leq \text{nums}[i] \leq 10$
- The product of any subarray of `nums` is **guaranteed** to fit in a **32-bit** integer.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def maxProduct(self, nums: List[int]) -> int:

        result = curr_min = curr_max = nums[0]

        for i in range(1, len(nums)):
            if nums[i] == 0:
                curr_max = 1
                curr_min = 1
                result = max(result, 0)
                continue
            prev_min, prev_max = curr_min, curr_max

            curr_max = max(nums[i], nums[i] * prev_max, nums[i] * prev_min)
            curr_min = min(nums[i], nums[i] * prev_max, nums[i] * prev_min)
            result = max(result, curr_max)

        return result
```

[229 Majority Element II \(link\)](#)

Description

Given an integer array of size n , find all elements that appear more than $\lfloor n/3 \rfloor$ times.

Example 1:

Input: `nums = [3,2,3]`
Output: `[3]`

Example 2:

Input: `nums = [1]`
Output: `[1]`

Example 3:

Input: `nums = [1,2]`
Output: `[1,2]`

Constraints:

- $1 \leq \text{nums.length} \leq 5 * 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

Follow up: Could you solve the problem in linear time and in $O(1)$ space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def majorityElement(self, nums: List[int]) -> List[int]:
        nums.sort()
        times = len(nums) // 3
        elements = []
        i = count = 0

        while i < len(nums):
            num = nums[i]
            count = 1
            while i + 1 < len(nums) and nums[i + 1] == num:
                i += 1
                count += 1
            if count > times:
                elements.append(num)
            i += 1
        return elements
```

[560 Subarray Sum Equals K \(link\)](#)

Description

Given an array of integers `nums` and an integer `k`, return *the total number of subarrays whose sum equals to k*.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Example 1:

Input: `nums = [1,1,1], k = 2`
Output: 2

Example 2:

Input: `nums = [1,2,3], k = 3`
Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 2 * 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$
- $-10^7 \leq k \leq 10^7$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def subarraySum(self, nums: List[int], k: int) -> int:

        lookup = {0:1}
        count = 0

        prefix = 0
        for num in nums:
            prefix += num
            if prefix - k in lookup:
                count += lookup[prefix - k]
            lookup[prefix] = lookup.get(prefix,0) + 1

        return count
```

[128 Longest Consecutive Sequence \(link\)](#)

Description

Given an unsorted array of integers `nums`, return *the length of the longest consecutive elements sequence*.

You must write an algorithm that runs in $O(n)$ time.

Example 1:

Input: `nums = [100,4,200,1,3,2]`

Output: 4

Explanation: The longest consecutive elements sequence is `[1, 2, 3, 4]`. Therefore

Example 2:

Input: `nums = [0,3,7,2,5,8,4,6,0,1]`

Output: 9

Example 3:

Input: `nums = [1,0,1,2]`

Output: 3

Constraints:

- $0 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def longestConsecutive(self, nums: List[int]) -> int:
        count = 0
        max_count = 0
        hashset = set(nums)

        for num in hashset:
            if num - 1 not in hashset:
                count = 1
                curr_num = num

                while curr_num + 1 in hashset:
                    count += 1
                    curr_num += 1
                max_count = max(max_count, count)

        return max_count
```

54 Spiral Matrix (link)

Description

Given an $m \times n$ matrix, return *all elements of the matrix in spiral order*.

Example 1:

1	→	2	→	3
4	→	5		↓
↑				↓
7	←	8	←	9

Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]

Example 2:

1	→	2	→	3	→	4
5	→	6	→	7		↓
↑						↓
9	←	10	←	11	←	12

Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]

Constraints:

- `m == matrix.length`
- `n == matrix[i].length`
- `1 <= m, n <= 10`
- `-100 <= matrix[i][j] <= 100`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def spiralOrder(self, matrix: List[List[int]]) -> List[int]:
        res = []
        left, right = 0, len(matrix[0]) - 1
        top, bottom = 0, len(matrix) - 1

        while top <= bottom and left <= right:
            for i in range(left, right + 1):
                res.append(matrix[top][i])
            top += 1

            for i in range(top, bottom + 1):
                res.append(matrix[i][right])
            right -= 1

            if top <= bottom:
                for i in range(right, left - 1, -1):
                    res.append(matrix[bottom][i])
                bottom -= 1
            if left <= right:
                for i in range(bottom, top - 1, -1):
                    res.append(matrix[i][left])
                left += 1

        return res
```

[2271 Rearrange Array Elements by Sign \(link\)](#)

Description

You are given a **0-indexed** integer array `nums` of **even** length consisting of an **equal** number of positive and negative integers.

You should return the array of `nums` such that the the array follows the given conditions:

1. Every **consecutive pair** of integers have **opposite signs**.
2. For all integers with the same sign, the **order** in which they were present in `nums` is **preserved**.
3. The rearranged array begins with a positive integer.

Return *the modified array after rearranging the elements to satisfy the aforementioned conditions*.

Example 1:

Input: `nums = [3,1,-2,-5,2,-4]`

Output: `[3,-2,1,-5,2,-4]`

Explanation:

The positive integers in `nums` are `[3,1,2]`. The negative integers are `[-2,-5,-4]`. The only possible way to rearrange them such that they satisfy all conditions is `[3,-2,1,-5,2,-4]`. Other ways such as `[1,-2,2,-5,3,-4]`, `[3,1,2,-2,-5,-4]`, `[-2,3,-5,1,-4,2]` are incor

Example 2:

Input: `nums = [-1,1]`

Output: `[1,-1]`

Explanation:

1 is the only positive integer and -1 the only negative integer in `nums`. So `nums` is rearranged to `[1,-1]`.

Constraints:

- $2 \leq \text{nums.length} \leq 2 * 10^5$
- `nums.length` is **even**
- $1 \leq |\text{nums}[i]| \leq 10^5$
- `nums` consists of **equal** number of positive and negative integers.

It is not required to do the modifications in-place.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def rearrangeArray(self, nums: List[int]) -> List[int]:
        positive = 0
        negative = 1
        temp = [0] * len(nums)

        for i, num in enumerate(nums):
            if nums[i] >= 0:
                temp[positive] = nums[i]
                positive += 2
            else:
                temp[negative] = nums[i]
                negative += 2
        return temp
```

[1 Two Sum \(link\)](#)

Description

Given an array of integers `nums` and an integer `target`, return *indices of the two numbers such that they add up to target*.

You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:

Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:

Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^4$
- $-10^9 \leq \text{nums}[i] \leq 10^9$
- $-10^9 \leq \text{target} \leq 10^9$
- **Only one valid answer exists.**

Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def twoSum(self, nums: List[int], target: int) -> List[int]:

        order = dict()
        for i, num in enumerate(nums):
            complement = target - num
            if complement in order:
                return [i, order[complement]]
            order[num] = i
```

[136 Single Number \(link\)](#)

Description

Given a **non-empty** array of integers `nums`, every element appears *twice* except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: `nums = [2,2,1]`

Output: 1

Example 2:

Input: `nums = [4,1,2,1,2]`

Output: 4

Example 3:

Input: `nums = [1]`

Output: 1

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-3 * 10^4 \leq \text{nums}[i] \leq 3 * 10^4$
- Each element in the array appears twice except for one element which appears only once.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def singleNumber(self, nums: List[int]) -> int:
        result = 0
        for num in nums:
            result ^= num
        return result
```

[485 Max Consecutive Ones \(link\)](#)

Description

Given a binary array `nums`, return *the maximum number of consecutive 1's in the array*.

Example 1:

Input: `nums = [1,1,0,1,1,1]`

Output: 3

Explanation: The first two digits or the last three digits are consecutive 1s. Th

Example 2:

Input: `nums = [1,0,1,1,0,1]`

Output: 2

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- `nums[i]` is either 0 or 1.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> int:
        maxc = curr = 0
        for num in nums:
            if num:
                curr += 1
            else:
                maxc = max(maxc, curr)
                curr = 0
        return max(maxc, curr)
```

[268 Missing Number \(link\)](#)

Description

Given an array `nums` containing n distinct numbers in the range $[0, n]$, return *the only number in the range that is missing from the array*.

Example 1:

Input: `nums = [3,0,1]`

Output: 2

Explanation:

$n = 3$ since there are 3 numbers, so all numbers are in the range $[0, 3]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 2:

Input: `nums = [0,1]`

Output: 2

Explanation:

$n = 2$ since there are 2 numbers, so all numbers are in the range $[0, 2]$. 2 is the missing number in the range since it does not appear in `nums`.

Example 3:

Input: `nums = [9,6,4,2,3,5,7,0,1]`

Output: 8

Explanation:

$n = 9$ since there are 9 numbers, so all numbers are in the range $[0, 9]$. 8 is the missing number in the range since it does not appear in `nums`.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 10^4$
- $0 \leq \text{nums}[i] \leq n$
- All the numbers of `nums` are **unique**.

Follow up: Could you implement a solution using only $O(1)$ extra space complexity and $O(n)$ runtime complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def missingNumber(self, nums: List[int]) -> int:
        n = len(nums)
        expected_sum = (n * (n + 1)) // 2
        actual_sum = sum(nums)
        return expected_sum - actual_sum
```

283 Move Zeroes [\(link\)](#)

Description

Given an integer array `nums`, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.

Example 1:

Input: `nums = [0,1,0,3,12]`
Output: `[1,3,12,0,0]`

Example 2:

Input: `nums = [0]`
Output: `[0]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$

Follow up: Could you minimize the total number of operations done?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def moveZeroes(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        j = 0
        for i in range(len(nums)):
            if nums[i] != 0:
                nums[j] = nums[i]
                j += 1

        for i in range(j, len(nums)):
            nums[i] = 0

        return
```

[189 Rotate Array \(link\)](#)

Description

Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.

Example 1:

Input: `nums = [1,2,3,4,5,6,7], k = 3`
Output: `[5,6,7,1,2,3,4]`
Explanation:
rotate 1 steps to the right: `[7,1,2,3,4,5,6]`
rotate 2 steps to the right: `[6,7,1,2,3,4,5]`
rotate 3 steps to the right: `[5,6,7,1,2,3,4]`

Example 2:

Input: `nums = [-1,-100,3,99], k = 2`
Output: `[3,99,-1,-100]`
Explanation:
rotate 1 steps to the right: `[99,-1,-100,3]`
rotate 2 steps to the right: `[3,99,-1,-100]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-2^{31} \leq \text{nums}[i] \leq 2^{31} - 1$
- $0 \leq k \leq 10^5$

Follow up:

- Try to come up with as many solutions as you can. There are at least **three** different ways to solve this problem.
- Could you do it in-place with $O(1)$ extra space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def rotate(self, nums: List[int], k: int) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        def reverse(start, end):
            while start < end:
                nums[start], nums[end] = nums[end], nums[start]
                start += 1
                end -= 1
        k = k % len(nums)
        reverse(0, len(nums) - 1)
        reverse(0, k - 1)
        reverse(k, len(nums) - 1)
        return
```

[1878 Check if Array Is Sorted and Rotated \(link\)](#)

Description

Given an array `nums`, return `true` *if the array was originally sorted in non-decreasing order, then rotated **some** number of positions (including zero)*. Otherwise, return `false`.

There may be **duplicates** in the original array.

Note: An array `A` rotated by `x` positions results in an array `B` of the same length such that `B[i] == A[(i+x) % A.length]` for every valid index `i`.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: `true`

Explanation: `[1,2,3,4,5]` is the original sorted array.

You can rotate the array by `x = 3` positions to begin on the element of value 3: `[`

Example 2:

Input: `nums = [2,1,3,4]`

Output: `false`

Explanation: There is no sorted array once rotated that can make `nums`.

Example 3:

Input: `nums = [1,2,3]`

Output: `true`

Explanation: `[1,2,3]` is the original sorted array.

You can rotate the array by `x = 0` positions (i.e. no rotation) to make `nums`.

Constraints:

- `1 <= nums.length <= 100`
- `1 <= nums[i] <= 100`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def check(self, nums: List[int]) -> bool:
        n = len(nums)
        count = 0

        for i in range(n-1):
            if nums[i] > nums[i + 1]:
                count += 1
        if nums[0] < nums[n-1]:
            count += 1

        return count <=1

    return True
```

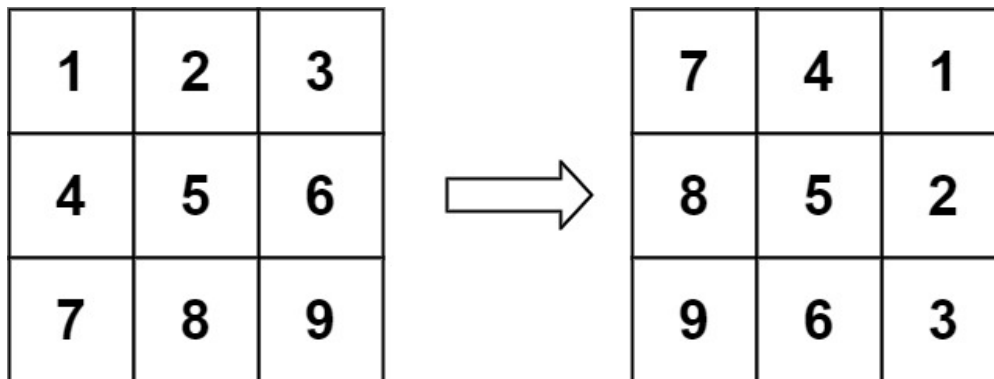
48 Rotate Image ([link](#))

Description

You are given an $n \times n$ 2D matrix representing an image, rotate the image by **90** degrees (clockwise).

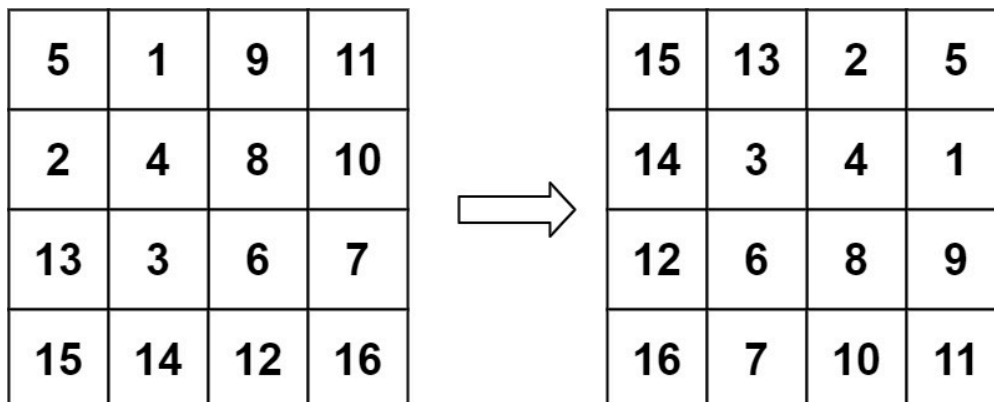
You have to rotate the image **in-place**, which means you have to modify the input 2D matrix directly. **DO NOT** allocate another 2D matrix and do the rotation.

Example 1:



Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]

Example 2:



Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]

Constraints:

- $n == \text{matrix.length} == \text{matrix}[i].\text{length}$

- $1 \leq n \leq 20$
- $-1000 \leq \text{matrix}[i][j] \leq 1000$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def rotate(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """

        for i in range(len(matrix)):
            for j in range(i+1, len(matrix[0])):
                matrix[i][j], matrix[j][i] = matrix[j][i], matrix[i][j]

        for row in matrix:
            row.reverse()
```

[75 Sort Colors \(link\)](#)

Description

Given an array `nums` with n objects colored red, white, or blue, sort them [in-place](#) so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers `0`, `1`, and `2` to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 300$
- `nums[i]` is either `0`, `1`, or `2`.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def sortColors(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        low = mid = 0
        high = len(nums) - 1

        while mid <= high:
            if nums[mid] == 0:
                nums[low] , nums[mid] = nums[mid], nums[low]
                mid += 1
                low += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[high] , nums[mid] = nums[mid], nums[high]
                high -= 1
```

[53 Maximum Subarray \(link\)](#)

Description

Given an integer array `nums`, find the subarray with the largest sum, and return *its sum*.

Example 1:

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]`
Output: 6
Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.

Example 2:

Input: `nums = [1]`
Output: 1
Explanation: The subarray `[1]` has the largest sum 1.

Example 3:

Input: `nums = [5,4,-1,7,8]`
Output: 23
Explanation: The subarray `[5,4,-1,7,8]` has the largest sum 23.

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

Follow up: If you have figured out the $O(n)$ solution, try coding another solution using the **divide and conquer** approach, which is more subtle.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def maxSubArray(self, nums: List[int]) -> int:
        max_sum = float('-inf')

        curr_sum = 0
        for num in nums:
            curr_sum += num
            max_sum = curr_sum if curr_sum > max_sum else max_sum

            if curr_sum < 0:
                curr_sum = 0

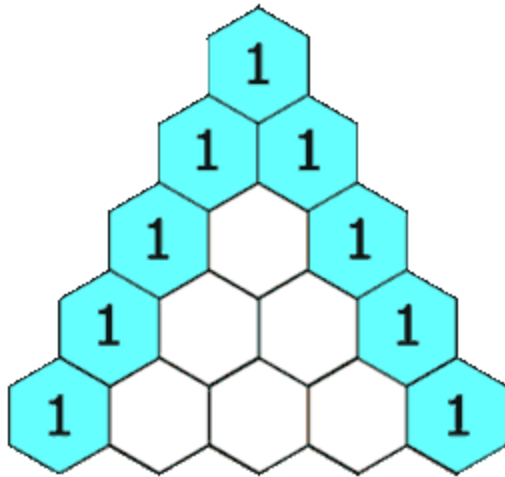
        return max_sum
```

[118 Pascal's Triangle \(link\)](#)

Description

Given an integer `numRows`, return the first `numRows` of **Pascal's triangle**.

In **Pascal's triangle**, each number is the sum of the two numbers directly above it as shown:



Example 1:

Input: `numRows = 5`
Output: `[[1], [1,1], [1,2,1], [1,3,3,1], [1,4,6,4,1]]`

Example 2:

Input: `numRows = 1`
Output: `[[1]]`

Constraints:

- $1 \leq \text{numRows} \leq 30$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def generate(self, numRows: int) -> List[List[int]]:
        pascal = [[1]]

        for i in range(1, numRows):
            temp = [1]
            for j in range(1, i):
                temp.append(pascal[i-1][j-1] + pascal[i-1][j])
            temp.append(1)
            pascal.append(temp)
        return pascal
```

[31 Next Permutation \(link\)](#)

Description

A **permutation** of an array of integers is an arrangement of its members into a sequence or linear order.

- For example, for `arr = [1,2,3]`, the following are all the permutations of `arr`:
`[1,2,3]`, `[1,3,2]`, `[2, 1, 3]`, `[2, 3, 1]`, `[3,1,2]`, `[3,2,1]`.

The **next permutation** of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the **next permutation** of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- For example, the next permutation of `arr = [1,2,3]` is `[1,3,2]`.
- Similarly, the next permutation of `arr = [2,3,1]` is `[3,1,2]`.
- While the next permutation of `arr = [3,2,1]` is `[1,2,3]` because `[3,2,1]` does not have a lexicographical larger rearrangement.

Given an array of integers `nums`, *find the next permutation of* `nums`.

The replacement must be [in place](#) and use only constant extra memory.

Example 1:

Input: `nums = [1,2,3]`
Output: `[1,3,2]`

Example 2:

Input: `nums = [3,2,1]`
Output: `[1,2,3]`

Example 3:

Input: `nums = [1,1,5]`
Output: `[1,5,1]`

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def nextPermutation(self, nums: List[int]) -> None:
        """
        Do not return anything, modify nums in-place instead.
        """
        # Step 1: Find the first decreasing element from the right
        i = len(nums) - 2

        while i >= 0 and nums[i] >= nums[i+1]:
            i -= 1

        j = len(nums) - 1

        if i >= 0:
            while nums[j] <= nums[i]:
                j -= 1
            nums[i], nums[j] = nums[j], nums[i]

        left, right = i + 1, len(nums) - 1
        while left <= right:
            nums[left], nums[right] = nums[right], nums[left]
            left += 1
            right -= 1
```

73 Set Matrix Zeroes (link)

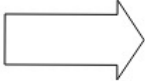
Description

Given an $m \times n$ integer matrix `matrix`, if an element is 0, set its entire row and column to 0's.

You must do it [in place](#).

Example 1:

1	1	1
1	0	1
1	1	1

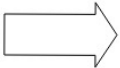


1	0	1
0	0	0
1	0	1

Input: `matrix = [[1,1,1],[1,0,1],[1,1,1]]`
Output: `[[1,0,1],[0,0,0],[1,0,1]]`

Example 2:

0	1	2	0
3	4	5	2
1	3	1	5



0	0	0	0
0	4	5	0
0	3	1	0

Input: `matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]`
Output: `[[0,0,0,0],[0,4,5,0],[0,3,1,0]]`

Constraints:

- `m == matrix.length`
- `n == matrix[0].length`
- `1 <= m, n <= 200`
- `-231 <= matrix[i][j] <= 231 - 1`

Follow up:

- A straightforward solution using $O(mn)$ space is probably a bad idea.
- A simple improvement uses $O(m + n)$ space, but still not the best solution.
- Could you devise a constant space solution?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def setZeroes(self, matrix: List[List[int]]) -> None:
        """
        Do not return anything, modify matrix in-place instead.
        """
        rmz = cmz = 0
        for i in range(len(matrix)):
            if not matrix[i][0]:
                cmz = 1
                break
        for j in range(len(matrix[0])):
            if not matrix[0][j]:
                rmz = 1
                break

        for i in range(1, len(matrix)):
            for j in range(1, len(matrix[0])):
                if not matrix[i][j]:
                    matrix[i][0] = 0
                    matrix[0][j] = 0

        for i in range(1, len(matrix)):
            for j in range(1, len(matrix[0])):
                if not matrix[i][0] or not matrix[0][j]:
                    matrix[i][j] = 0

        for i in range(0, len(matrix)):
            if cmz:
                matrix[i][0] = 0
        for j in range(0, len(matrix[0])):
            if rmz:
                matrix[0][j] = 0
```

[207 Course Schedule \(link\)](#)

Description

There are a total of `numCourses` courses you have to take, labeled from `0` to `numCourses - 1`. You are given an array `prerequisites` where `prerequisites[i] = [ai, bi]` indicates that you **must** take course `bi` first if you want to take course `ai`.

- For example, the pair `[0, 1]`, indicates that to take course `0` you have to first take course `1`.

Return `true` if you can finish all courses. Otherwise, return `false`.

Example 1:

Input: `numCourses = 2, prerequisites = [[1,0]]`

Output: `true`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0. So it is possible.

Example 2:

Input: `numCourses = 2, prerequisites = [[1,0],[0,1]]`

Output: `false`

Explanation: There are a total of 2 courses to take.

To take course 1 you should have finished course 0, and to take course 0 you should have finished course 1. So it is impossible.

Constraints:

- `1 <= numCourses <= 2000`
- `0 <= prerequisites.length <= 5000`
- `prerequisites[i].length == 2`
- `0 <= ai, bi < numCourses`
- All the pairs `prerequisites[i]` are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
from collections import defaultdict

class Solution:
    def canFinish(self, numCourses: int, prerequisites: List[List[int]]) -> bool:
        graph = defaultdict(list)
        in_degree = [0] * numCourses

        for c,p in prerequisites:
            graph[p].append(c)
            in_degree[c] += 1

        queue = deque([i for i in range(numCourses) if in_degree[i] == 0 ])
        count = 0

        while queue:
            course = queue.pop()
            count += 1
            for neighbour in graph[course]:
                in_degree[neighbour] -=1
                if in_degree[neighbour] == 0:
                    queue.append(neighbour)

        return count == numCourses
```

[215 Kth Largest Element in an Array \(link\)](#)

Description

Given an integer array `nums` and an integer `k`, return *the* k^{th} *largest element in the array*.

Note that it is the k^{th} largest element in the sorted order, not the k^{th} distinct element.

Can you solve it without sorting?

Example 1:

Input: `nums = [3,2,1,5,6,4]`, `k = 2`
Output: 5

Example 2:

Input: `nums = [3,2,3,1,2,4,5,5,6]`, `k = 4`
Output: 4

Constraints:

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $-10^4 \leq \text{nums}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
import heapq

class Solution:
    def findKthLargest(self, nums: List[int], k: int) -> int:
        k = len(nums) - k
        def qs(left, right):
            pivot = nums[random.randint(left, right)]
            l, r = left, right

            while l <= r:
                while nums[l] < pivot:
                    l += 1
                while nums[r] > pivot:
                    r -= 1
                if l <= r:
                    nums[l], nums[r] = nums[r], nums[l]
                    l += 1
                    r -= 1
            if k <= r:
                return qs(left, r)
            elif k >= l:
                return qs(l, right)
            else:
                return nums[k]

        return qs(0, len(nums) - 1)
```

[789 Kth Largest Element in a Stream \(link\)](#)

Description

You are part of a university admissions office and need to keep track of the k th highest test score from applicants in real-time. This helps to determine cut-off marks for interviews and admissions dynamically as new applicants submit their scores.

You are tasked to implement a class which, for a given integer k , maintains a stream of test scores and continuously returns the k th highest test score **after** a new score has been submitted. More specifically, we are looking for the k th highest score in the sorted list of all scores.

Implement the `KthLargest` class:

- `KthLargest(int k, int[] nums)` Initializes the object with the integer k and the stream of test scores `nums`.
- `int add(int val)` Adds a new test score `val` to the stream and returns the element representing the k^{th} largest element in the pool of test scores so far.

Example 1:

Input:

```
["KthLargest", "add", "add", "add", "add", "add"]  
[[3, [4, 5, 8, 2]], [3], [5], [10], [9], [4]]
```

Output: [null, 4, 5, 5, 8, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(3, [4, 5, 8, 2]);  
kthLargest.add(3); // return 4  
kthLargest.add(5); // return 5  
kthLargest.add(10); // return 5  
kthLargest.add(9); // return 8  
kthLargest.add(4); // return 8
```

Example 2:

Input:

```
["KthLargest", "add", "add", "add", "add"]  
[[4, [7, 7, 7, 7, 8, 3]], [2], [10], [9], [9]]
```

Output: [null, 7, 7, 7, 8]

Explanation:

```
KthLargest kthLargest = new KthLargest(4, [7, 7, 7, 7, 8, 3]);  
kthLargest.add(2); // return 7  
kthLargest.add(10); // return 7
```

```
kthLargest.add(9); // return 7  
kthLargest.add(9); // return 8
```

Constraints:

- $0 \leq \text{nums.length} \leq 10^4$
- $1 \leq k \leq \text{nums.length} + 1$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- $-10^4 \leq \text{val} \leq 10^4$
- At most 10^4 calls will be made to add.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
import heapq

class KthLargest:

    def __init__(self, k: int, nums: List[int]):
        self.k = k
        self.min_heap = nums
        heapq.heapify(self.min_heap)

        while(len(self.min_heap) > k):
            heapq.heappop(self.min_heap)

    def add(self, val: int) -> int:
        if len(self.min_heap) < self.k:
            heapq.heappush(self.min_heap, val)
        elif val > self.min_heap[0]:
            heapq.heapreplace(self.min_heap, val)

        return self.min_heap[0]

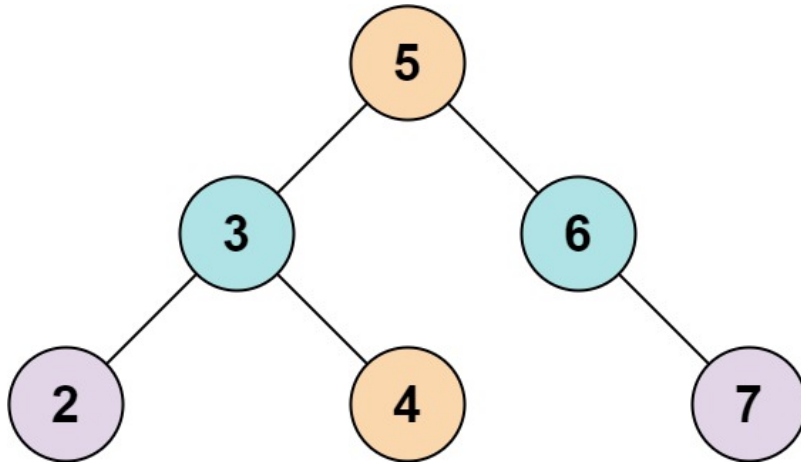
# Your KthLargest object will be instantiated and called as such:
# obj = KthLargest(k, nums)
# param_1 = obj.add(val)
```

653 Two Sum IV - Input is a BST (link)

Description

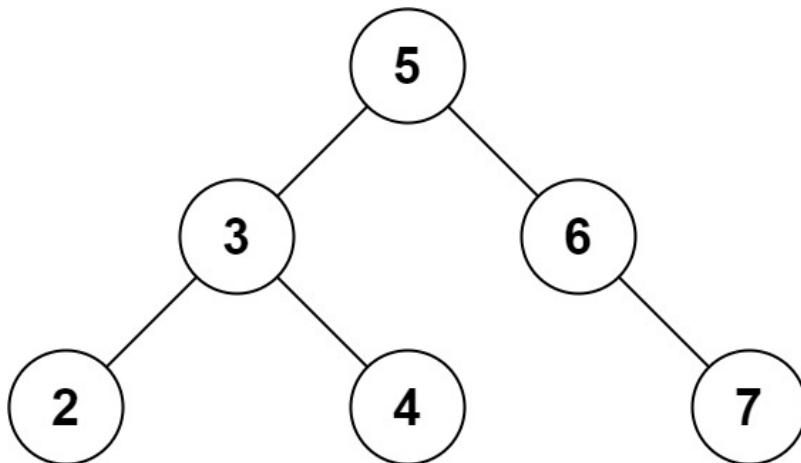
Given the root of a binary search tree and an integer k , return `true` *if there exist two elements in the BST such that their sum is equal to k* , or `false` otherwise.

Example 1:



Input: `root = [5,3,6,2,4,null,7]`, `k = 9`
Output: `true`

Example 2:



Input: `root = [5,3,6,2,4,null,7]`, `k = 28`
Output: `false`

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$
- root is guaranteed to be a **valid** binary search tree.
- $-10^5 \leq k \leq 10^5$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

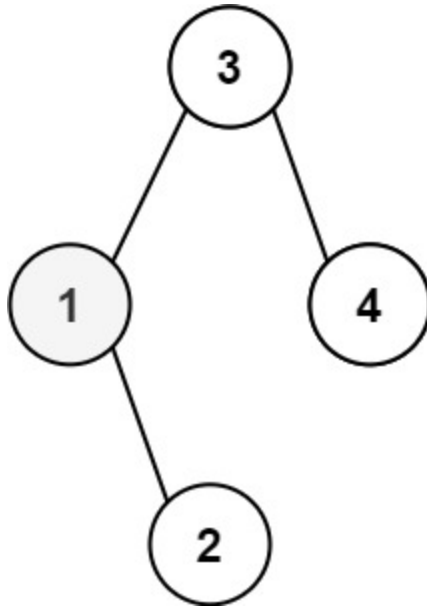
```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def findTarget(self, root: TreeNode, k: int) -> bool:
        seen = set()
        queue = deque([root])
        while queue:
            node = queue.popleft()
            if k - node.val in seen:
                return True
            seen.add(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        return False
```

[230 Kth Smallest Element in a BST \(link\)](#)

Description

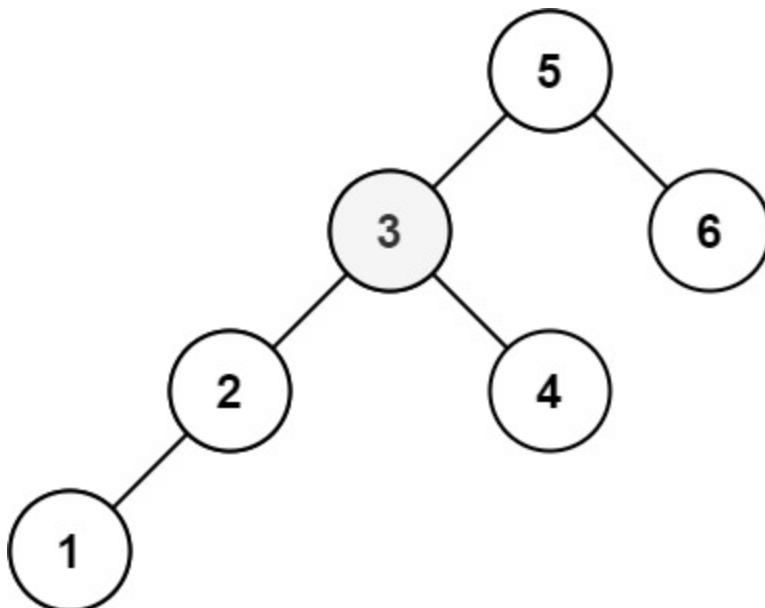
Given the root of a binary search tree, and an integer k , return *the k^{th} smallest value (1-indexed) of all the values of the nodes in the tree.*

Example 1:



Input: root = [3,1,4,null,2], k = 1
Output: 1

Example 2:



Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3

Constraints:

- The number of nodes in the tree is n.
- $1 \leq k \leq n \leq 10^4$
- $0 \leq \text{Node.val} \leq 10^4$

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def kthSmallest(self, root: Optional[TreeNode], k: int) -> int:
        self.k = k

        def recur(root):
            if not root:
                return None

            left = recur(root.left)
            if left is not None:
                return left

            self.k -= 1
            if self.k == 0:
                return root.val

            return recur(root.right)
        return recur(root)
```

1050 Construct Binary Search Tree from Preorder Traversal (link)

Description

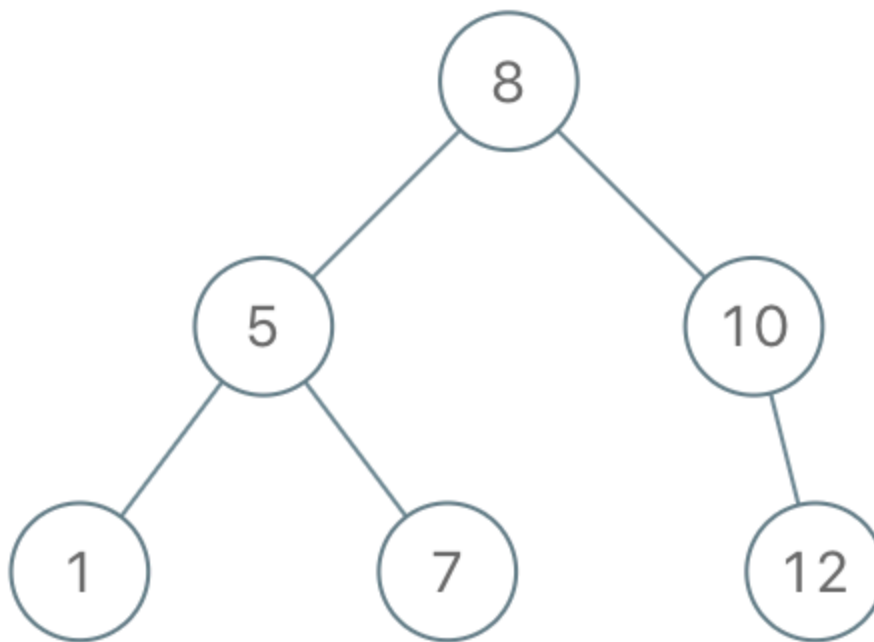
Given an array of integers `preorder`, which represents the **preorder traversal** of a BST (i.e., **binary search tree**), construct the tree and return *its root*.

It is **guaranteed** that there is always possible to find a binary search tree with the given requirements for the given test cases.

A **binary search tree** is a binary tree where for every node, any descendant of `Node.left` has a value **strictly less than** `Node.val`, and any descendant of `Node.right` has a value **strictly greater than** `Node.val`.

A **preorder traversal** of a binary tree displays the value of the node first, then traverses `Node.left`, then traverses `Node.right`.

Example 1:



Input: `preorder = [8,5,1,7,10,12]`

Output: `[8,5,10,1,7,null,12]`

Example 2:

Input: preorder = [1,3]
Output: [1,null,3]

Constraints:

- $1 \leq \text{preorder.length} \leq 100$
- $1 \leq \text{preorder}[i] \leq 1000$
- All the values of preorder are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def bstFromPreorder(self, preorder: List[int]) -> Optional[TreeNode]:
        self.index = 0
        def build(minv, maxv):
            if self.index >= len(preorder) or not(minv < preorder[self.index] < maxv):
                return None

            node = TreeNode(preorder[self.index])
            self.index += 1
            node.left = build(minv, node.val)
            node.right = build(node.val, maxv)

            return node
        return build(float('-inf'), float('inf'))
```

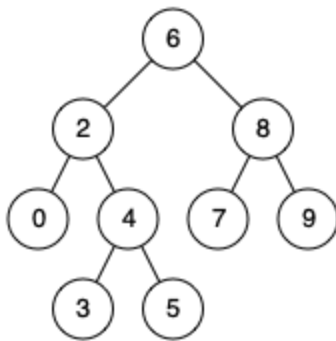
[235 Lowest Common Ancestor of a Binary Search Tree](#) (link)

Description

Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:

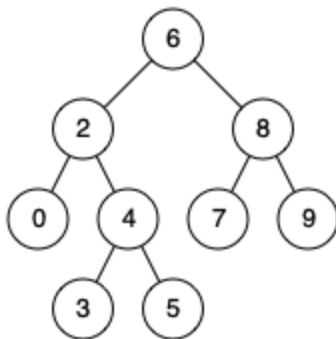


Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

Output: 6

Explanation: The LCA of nodes 2 and 8 is 6.

Example 2:



Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

Output: 2

Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself.

Example 3:

Input: root = [2,1], p = 2, q = 1
Output: 2

Constraints:

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$
- All `Node.val` are **unique**.
- $p \neq q$
- p and q will exist in the BST.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        if root.val >= min(p.val, q.val) and root.val <= max(p.val, q.val):
            return root

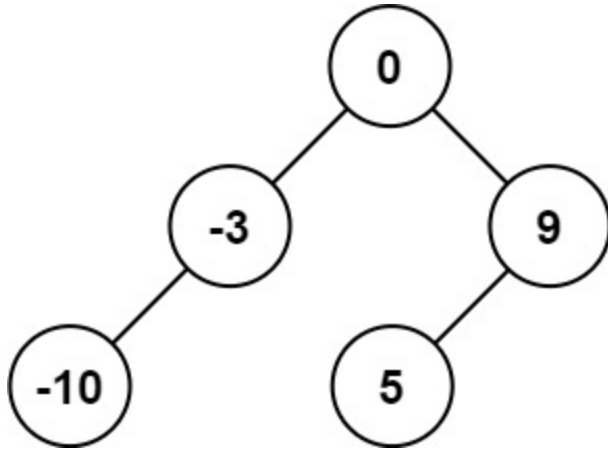
        if root.val < min(p.val, q.val):
            return self.lowestCommonAncestor(root.right, p, q)
        elif root.val > max(p.val, q.val):
            return self.lowestCommonAncestor(root.left, p, q)
```

[108 Convert Sorted Array to Binary Search Tree \(link\)](#)

Description

Given an integer array `nums` where the elements are sorted in **ascending order**, convert it to a **height-balanced** binary search tree.

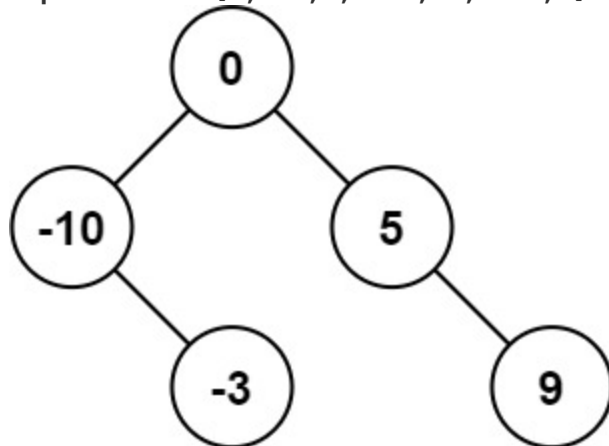
Example 1:



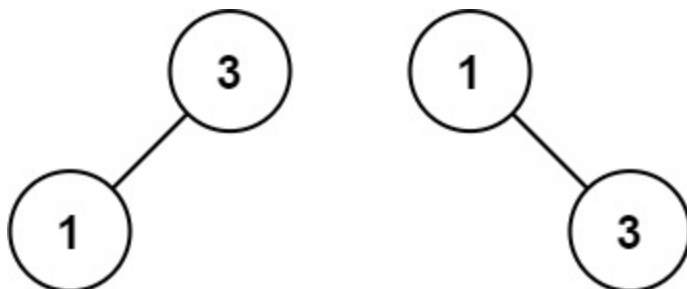
Input: `nums = [-10,-3,0,5,9]`

Output: `[0,-3,9,-10,null,5]`

Explanation: `[0,-10,5,null,-3,null,9]` is also accepted:



Example 2:



Input: `nums = [1,3]`

Output: `[3,1]`

Explanation: `[1,null,3]` and `[3,1]` are both height-balanced BSTs.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in a **strictly increasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def sortedArrayToBST(self, nums: List[int]) -> Optional[TreeNode]:

        def recur(half):
            if not half:
                return None

            idx = len(half)//2
            node = TreeNode(half[idx])

            node.left = recur(half[: idx])
            node.right = recur(half[idx+1:])
            return node
        return recur(nums)
```

[98 Validate Binary Search Tree \(link\)](#)

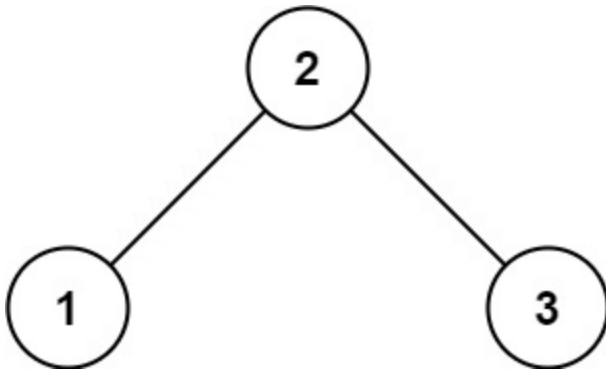
Description

Given the root of a binary tree, *determine if it is a valid binary search tree (BST)*.

A **valid BST** is defined as follows:

- The left subtree of a node contains only nodes with keys **strictly less than** the node's key.
- The right subtree of a node contains only nodes with keys **strictly greater than** the node's key.
- Both the left and right subtrees must also be binary search trees.

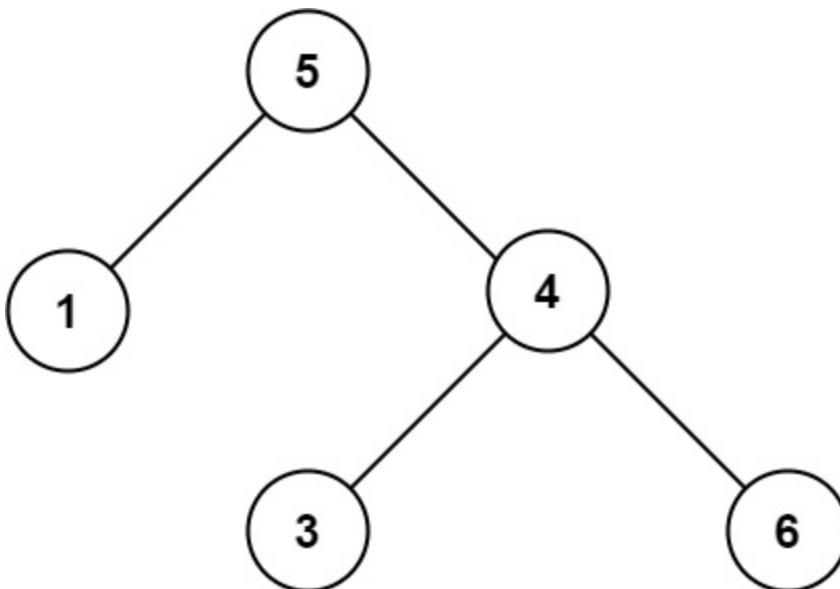
Example 1:



Input: root = [2,1,3]

Output: true

Example 2:



Input: root = [5,1,4,null,null,3,6]**Output:** false**Explanation:** The root node's value is 5 but its right child's value is 4.**Constraints:**

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-2^{31} \leq \text{Node.val} \leq 2^{31} - 1$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def recur(root, minv = float('-inf') , maxv = float('inf')):
            if not root:
                return True
            if not minv < root.val < maxv:
                return False
            return recur(root.left, minv, root.val) and recur(root.right, root.val, maxv)
        return recur(root)
```

116 Populating Next Right Pointers in Each Node (link)

Description

You are given a **perfect binary tree** where all leaves are on the same level, and every parent has two children. The binary tree has the following definition:

```
struct Node {
    int val;
    Node *left;
    Node *right;
    Node *next;
}
```

Populate each next pointer to point to its next right node. If there is no next right node, the next pointer should be set to NULL.

Initially, all next pointers are set to NULL.

Example 1:

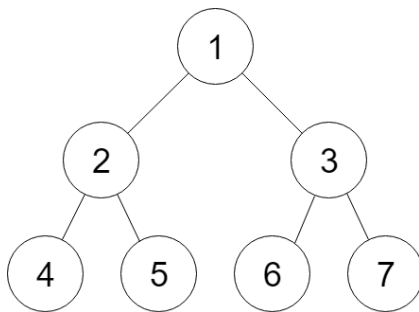


Figure A

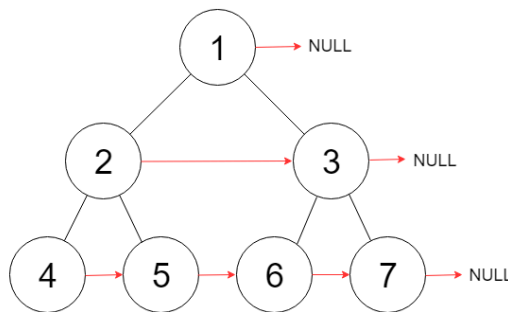


Figure B

Input: root = [1,2,3,4,5,6,7]

Output: [1,#,2,3,#,4,5,6,7,#]

Explanation: Given the above perfect binary tree (Figure A), your function should

Example 2:

Input: root = []

Output: []

Constraints:

- The number of nodes in the tree is in the range $[0, 2^{12} - 1]$.
- $-1000 \leq \text{Node.val} \leq 1000$

Follow-up:

- You may only use constant extra space.
- The recursive approach is fine. You may assume implicit stack space does not count as extra space for this problem.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def connect(self, root: 'Optional[Node]') -> 'Optional[Node]':
        if not root:
            return None

        leftmost = root

        while leftmost.left:
            head = leftmost
            while head:
                head.left.next = head.right
                if head.next:
                    head.right.next = head.next.left
                head = head.next
            leftmost = leftmost.left
        return root
```

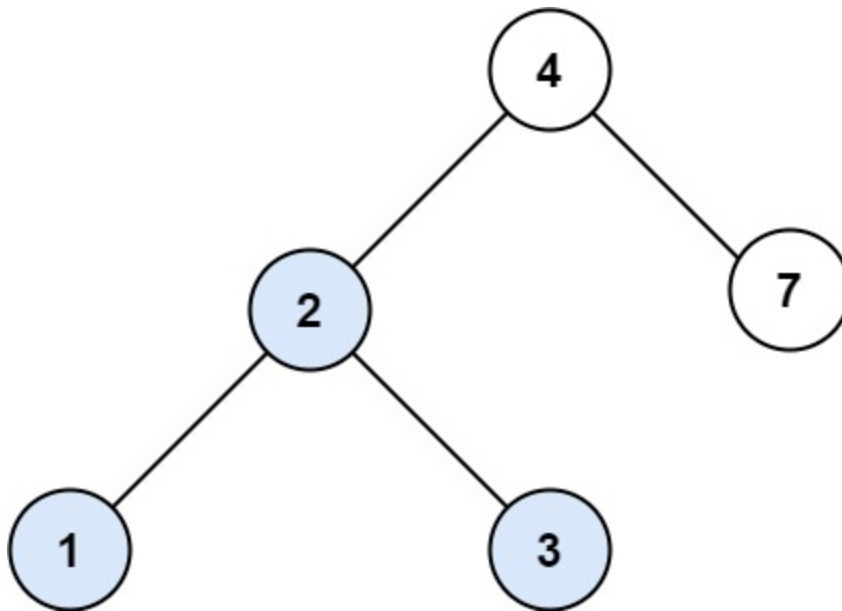
[783 Search in a Binary Search Tree \(link\)](#)

Description

You are given the root of a binary search tree (BST) and an integer `val`.

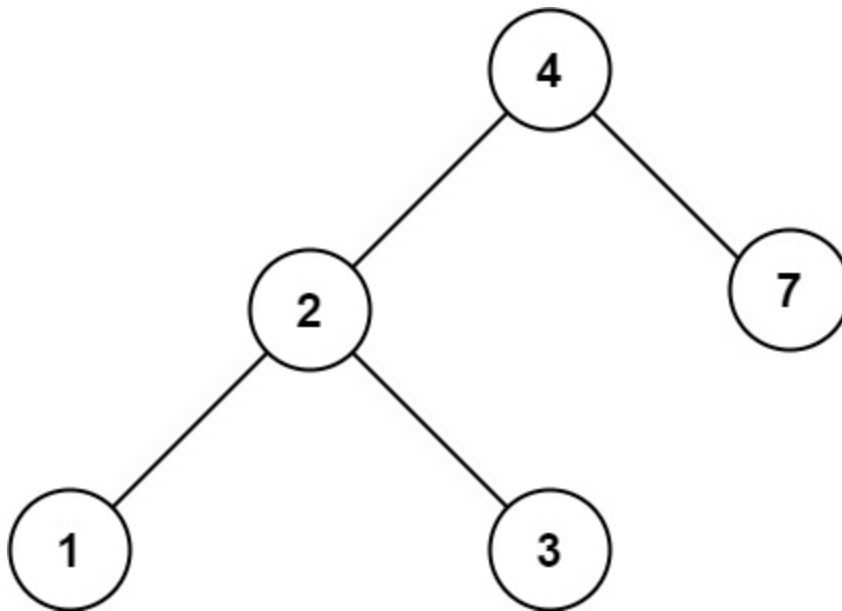
Find the node in the BST that the node's value equals `val` and return the subtree rooted with that node. If such a node does not exist, return `null`.

Example 1:



Input: `root = [4,2,7,1,3]`, `val = 2`
Output: `[2,1,3]`

Example 2:



Input: root = [4,2,7,1,3], val = 5

Output: []

Constraints:

- The number of nodes in the tree is in the range [1, 5000].
- $1 \leq \text{Node.val} \leq 10^7$
- root is a binary search tree.
- $1 \leq \text{val} \leq 10^7$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

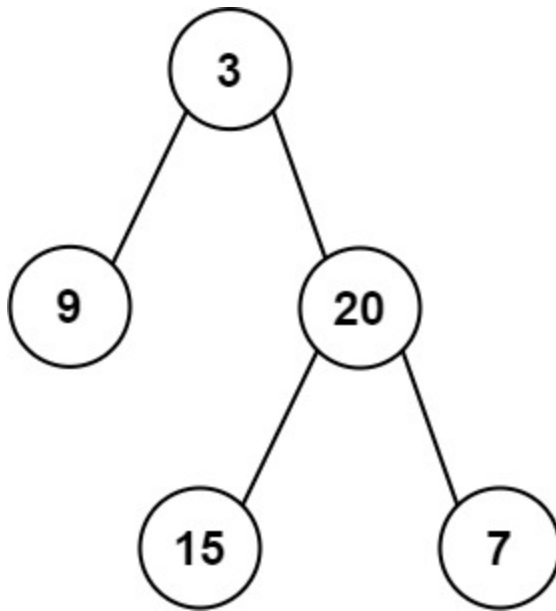
```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def searchBST(self, root: Optional[TreeNode], val: int) -> Optional[TreeNode]:
        while root:
            if root.val == val:
                return root
            elif root.val > val:
                root = root.left
            else:
                root = root.right
        return None
```

105 Construct Binary Tree from Preorder and Inorder Traversal (link)

Description

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: `preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`
Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `preorder = [-1]`, `inorder = [-1]`
Output: `[-1]`

Constraints:

- $1 \leq \text{preorder.length} \leq 3000$
- $\text{inorder.length} == \text{preorder.length}$
- $-3000 \leq \text{preorder}[i], \text{inorder}[i] \leq 3000$
- `preorder` and `inorder` consist of **unique** values.
- Each value of `inorder` also appears in `preorder`.
- `preorder` is **guaranteed** to be the preorder traversal of the tree.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def buildTree(self, preorder: List[int], inorder: List[int]) -> Optional[TreeNode]:
        # Build value-to-index map for inorder traversal
        val_to_index = {val: i for i, val in enumerate(inorder)}
        self.pre_idx = 0

        def helper(in_left, in_right):
            if in_left > in_right:
                return None

            # Pick current root from preorder
            root_val = preorder[self.pre_idx]
            self.pre_idx += 1
            root = TreeNode(root_val)

            # Split inorder by index
            idx = val_to_index[root_val]
            root.left = helper(in_left, idx - 1)
            root.right = helper(idx + 1, in_right)

            return root

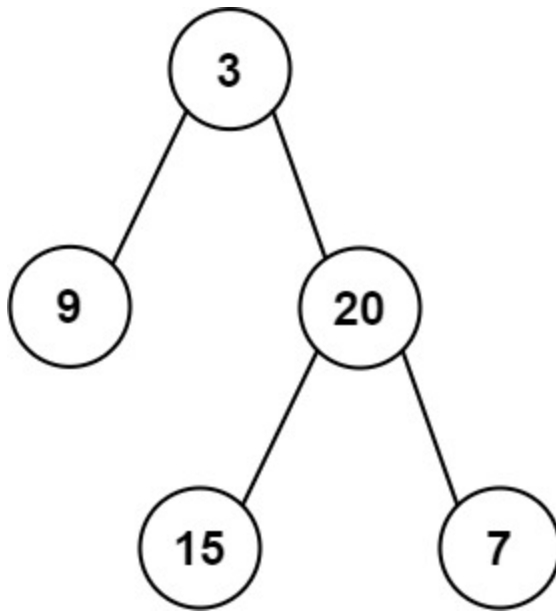
        return helper(0, len(inorder) - 1)
```

106 Construct Binary Tree from Inorder and Postorder Traversal (link)

Description

Given two integer arrays `inorder` and `postorder` where `inorder` is the inorder traversal of a binary tree and `postorder` is the postorder traversal of the same tree, construct and return *the binary tree*.

Example 1:



Input: `inorder = [9,3,15,20,7]`, `postorder = [9,15,7,20,3]`
Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `inorder = [-1]`, `postorder = [-1]`
Output: `[-1]`

Constraints:

- `1 <= inorder.length <= 3000`
- `postorder.length == inorder.length`
- `-3000 <= inorder[i], postorder[i] <= 3000`
- `inorder` and `postorder` consist of **unique** values.
- Each value of `postorder` also appears in `inorder`.
- `inorder` is **guaranteed** to be the inorder traversal of the tree.
- `postorder` is **guaranteed** to be the postorder traversal of the tree.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def buildTree(self, inorder: List[int], postorder: List[int]) -> Optional[TreeNode]:
        if not postorder or not inorder:
            return None

        val = postorder.pop()
        root = TreeNode(val)

        index = inorder.index(val)

        root.right = self.buildTree(inorder[index + 1:], postorder)
        root.left = self.buildTree(inorder[:index], postorder)

        return root
```

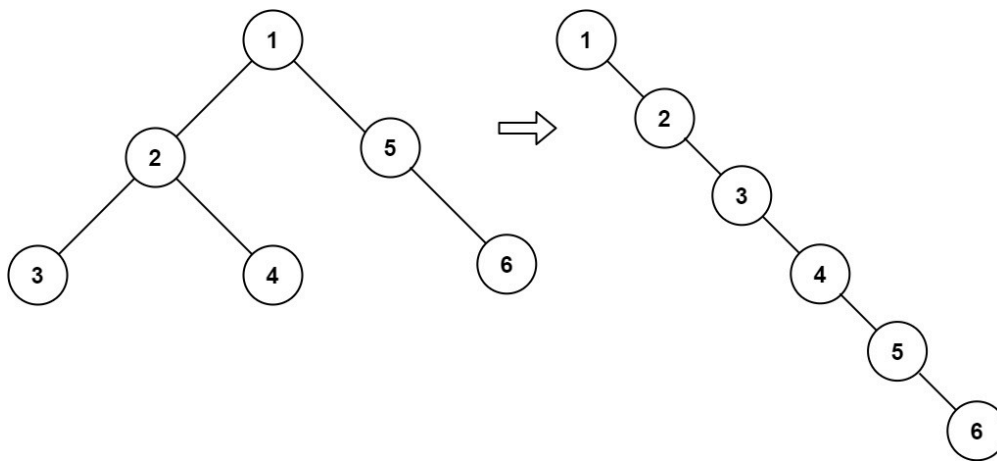
114 Flatten Binary Tree to Linked List (link)

Description

Given the root of a binary tree, flatten the tree into a "linked list":

- The "linked list" should use the same `TreeNode` class where the `right` child pointer points to the next node in the list and the `left` child pointer is always `null`.
- The "linked list" should be in the same order as a **pre-order traversal** of the binary tree.

Example 1:



Input: `root = [1,2,5,3,4,null,6]`

Output: `[1,null,2,null,3,null,4,null,5,null,6]`

Example 2:

Input: `root = []`

Output: `[]`

Example 3:

Input: `root = [0]`

Output: `[0]`

Constraints:

- The number of nodes in the tree is in the range `[0, 2000]`.
- `-100 <= Node.val <= 100`

Follow up: Can you flatten the tree in-place (with $O(1)$ extra space)?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def flatten(self, root: Optional[TreeNode]) -> None:
        """
        Do not return anything, modify root in-place instead.
        """
        if not root:
            return
        stack = [root]
        prev = None

        while stack:
            curr = stack.pop()

            if prev:
                prev.left = None
                prev.right = curr

            if curr.right:
                stack.append(curr.right)
            if curr.left:
                stack.append(curr.left)
            prev = curr

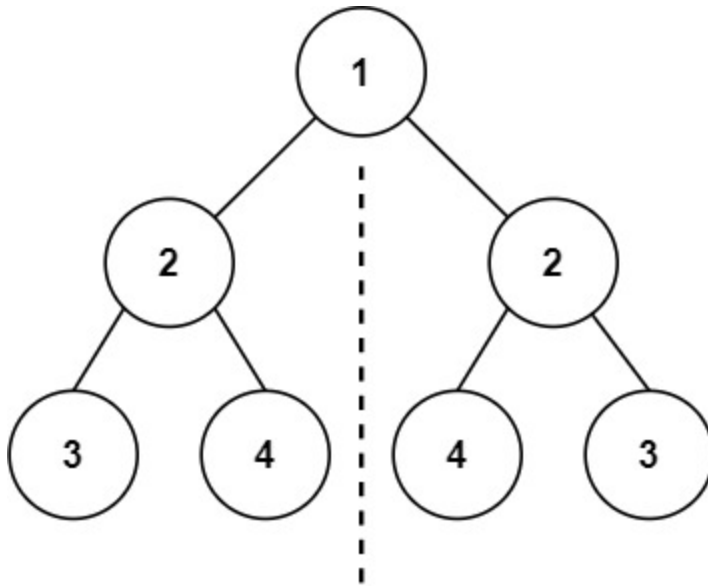
        return
```

[101 Symmetric Tree \(link\)](#)

Description

Given the root of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).

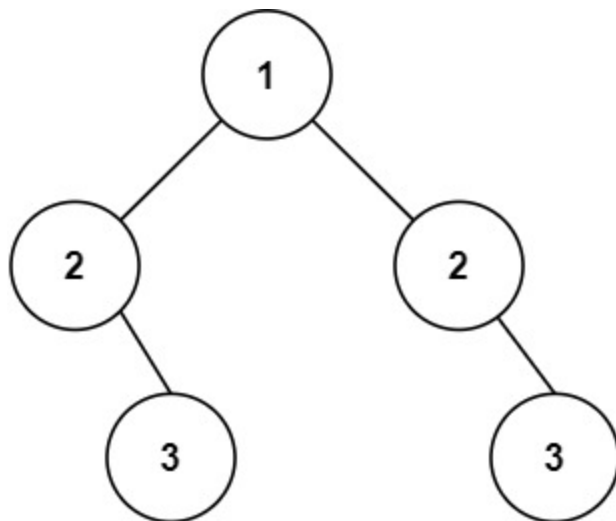
Example 1:



Input: root = [1,2,2,3,4,4,3]

Output: true

Example 2:



Input: root = [1,2,2,null,3,null,3]

Output: false

Constraints:

- The number of nodes in the tree is in the range $[1, 1000]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Could you solve it both recursively and iteratively?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        if not root:
            return True # A null tree is symmetric

        def recur(t1, t2):
            if not t1 and not t2:
                return True
            if not t1 or not t2:
                return False
            if t1.val != t2.val:
                return False
            return recur(t1.left, t2.right) and recur(t1.right, t2.left)

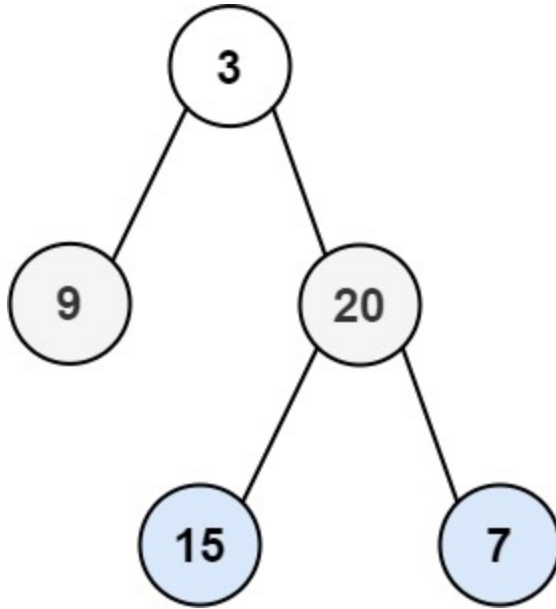
        return recur(root.left, root.right)
```

[103 Binary Tree Zigzag Level Order Traversal \(link\)](#)

Description

Given the root of a binary tree, return *the zigzag level order traversal of its nodes' values*. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def zigzagLevelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:

        if not root:
            return []

        queue = deque([root])
        ltr = True
        output = []

        while queue:
            length = len(queue)
            curr_list = []

            for _ in range(length):
                node = queue.popleft()
                curr_list.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            if not ltr:
                output.append(curr_list[::-1])
            else:
                output.append(curr_list)
            ltr = not ltr
        return output
```

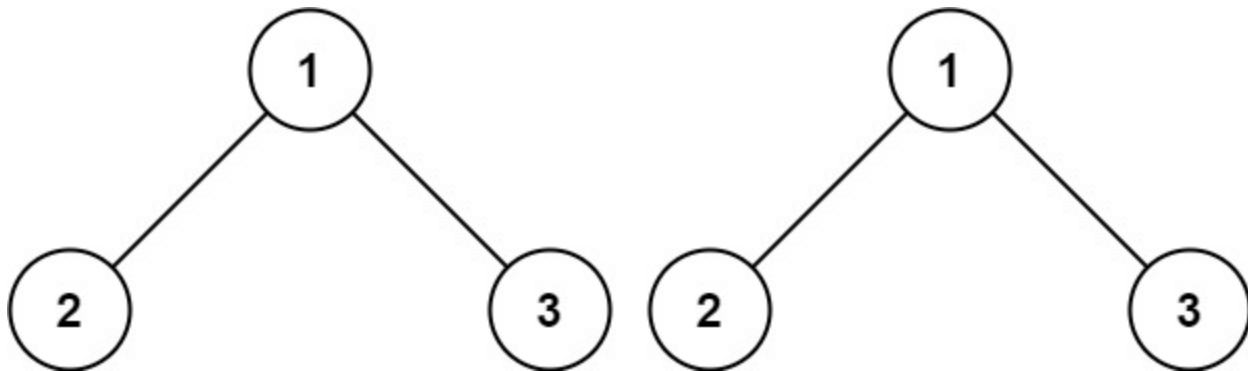
[100 Same Tree \(link\)](#)

Description

Given the roots of two binary trees p and q , write a function to check if they are the same or not.

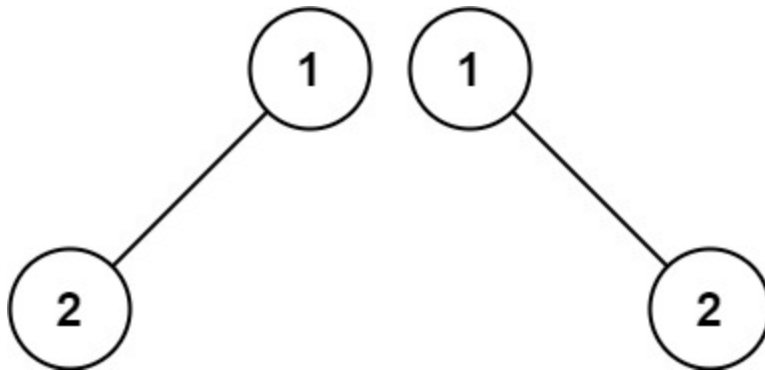
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



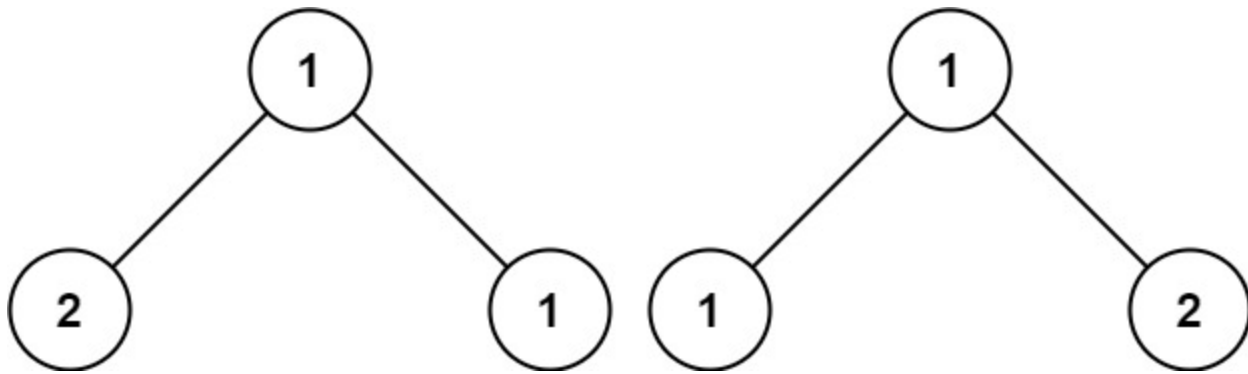
Input: $p = [1,2,3]$, $q = [1,2,3]$
Output: true

Example 2:



Input: $p = [1,2]$, $q = [1,null,2]$
Output: false

Example 3:



Input: p = [1,2,1], q = [1,1,2]
Output: false

Constraints:

- The number of nodes in both trees is in the range $[0, 100]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:

        def recur(p,q):
            if not q and not p:
                return True
            if not q or not p:
                return False
            if q.val != p.val:
                return False
            return recur(p.left, q.left) and recur(p.right, q.right)
        return recur(p,q)
```

1218 Lowest Common Ancestor of Deepest Leaves ([link](#))

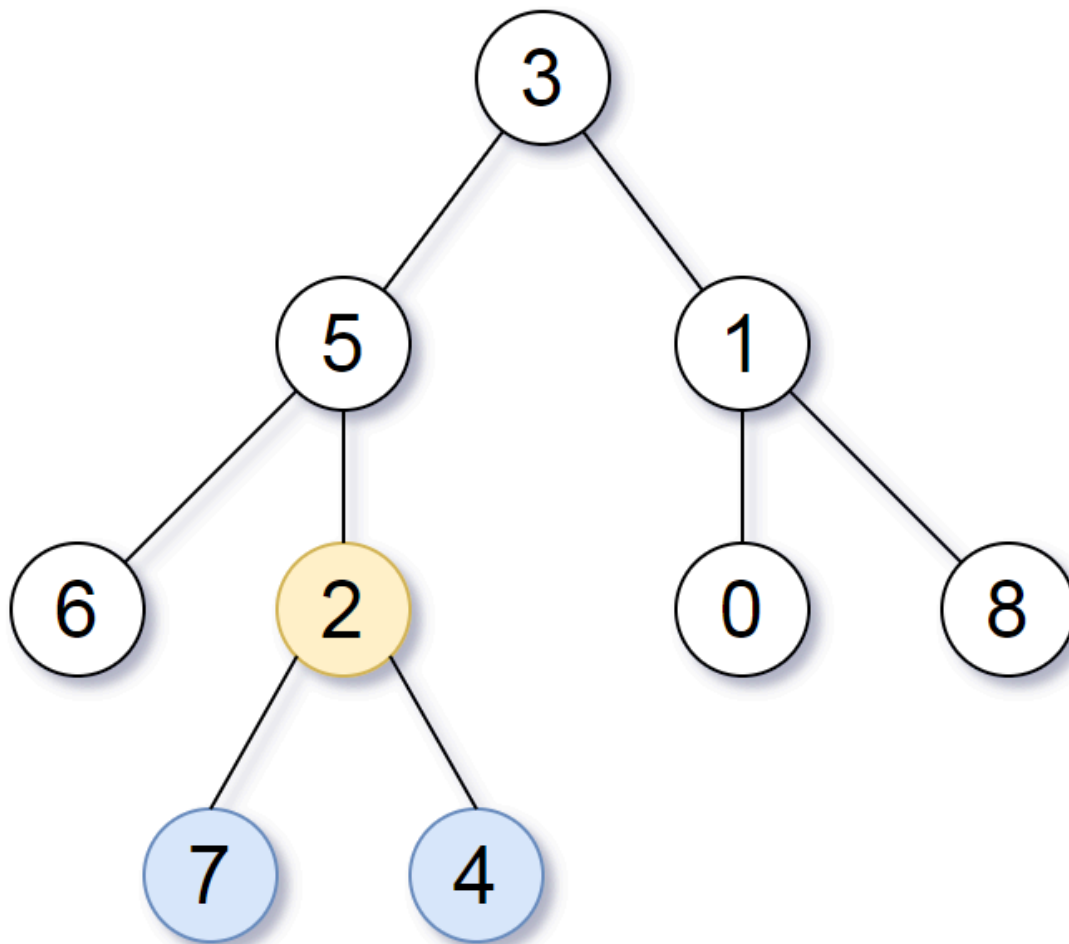
Description

Given the root of a binary tree, return *the lowest common ancestor of its deepest leaves*.

Recall that:

- The node of a binary tree is a leaf if and only if it has no children
- The depth of the root of the tree is 0. If the depth of a node is d , the depth of each of its children is $d + 1$.
- The lowest common ancestor of a set S of nodes, is the node A with the largest depth such that every node in S is in the subtree with root A .

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest leaf-nodes of the tree.
Note that nodes 6, 0, and 8 are also leaf nodes, but the depth of them is 2, but

Example 2:

Input: root = [1]

Output: [1]

Explanation: The root is the deepest node in the tree, and it's the lca of itself

Example 3:

Input: root = [0,1,3,null,2]

Output: [2]

Explanation: The deepest leaf node in the tree is 2, the lca of one node is itself

Constraints:

- The number of nodes in the tree will be in the range [1, 1000].
- $0 \leq \text{Node.val} \leq 1000$
- The values of the nodes in the tree are **unique**.

Note: This question is the same as 865: <https://leetcode.com/problems/smallest-subtree-with-all-the-deepest-nodes/>

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def lcaDeepestLeaves(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def recur(root):
            if not root:
                return(0, None)

            ld, left = recur(root.left)
            rd, right = recur(root.right)

            if ld == rd:
                return (ld +1, root)
            elif ld > rd:
                return (ld+1, left)
            else:
                return (rd + 1, right)

        return recur(root)[1]
```

896 Smallest Subtree with all the Deepest Nodes (link)

Description

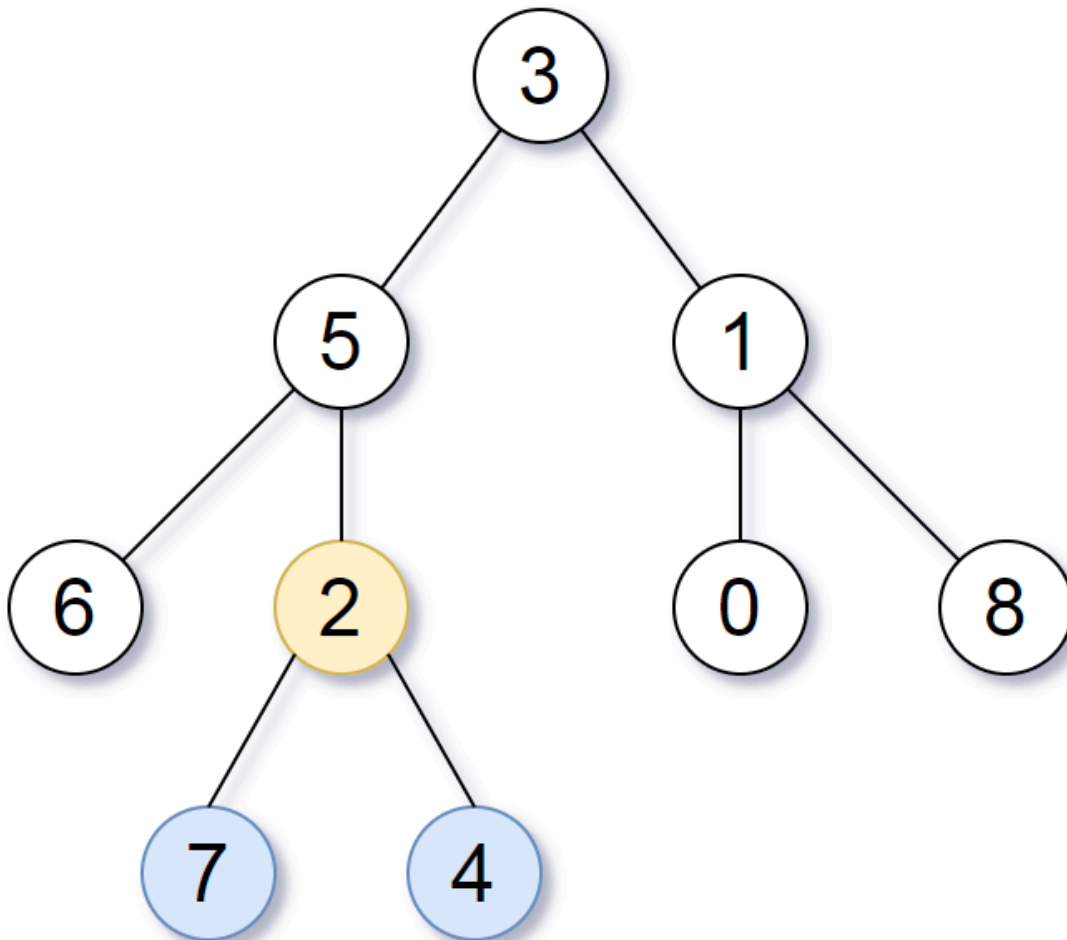
Given the root of a binary tree, the depth of each node is **the shortest distance to the root**.

Return *the smallest subtree* such that it contains **all the deepest nodes** in the original tree.

A node is called **the deepest** if it has the largest depth possible among any node in the entire tree.

The **subtree** of a node is a tree consisting of that node, plus the set of all descendants of that node.

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4]

Output: [2,7,4]

Explanation: We return the node with value 2, colored in yellow in the diagram.

The nodes coloured in blue are the deepest nodes of the tree.
Notice that nodes 5, 3 and 2 contain the deepest nodes in the tree but node 2 is

Example 2:

Input: root = [1]

Output: [1]

Explanation: The root is the deepest node in the tree.

Example 3:

Input: root = [0,1,3,null,2]

Output: [2]

Explanation: The deepest node in the tree is 2, the valid subtrees are the subtree

Constraints:

- The number of nodes in the tree will be in the range [1, 500].
- $0 \leq \text{Node.val} \leq 500$
- The values of the nodes in the tree are **unique**.

Note: This question is the same as 1123: <https://leetcode.com/problems/lowest-common-ancestor-of-deepest-leaves/>

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def subtreeWithAllDeepest(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        def recur(root):
            if not root:
                return (0, None)

            left_depth, left_node = recur(root.left)
            right_depth, right_node = recur(root.right)

            if left_depth == right_depth:
                return (left_depth + 1, root)
            elif left_depth > right_depth:
                return (left_depth + 1, left_node)
            else:
                return (right_depth + 1, right_node)

        return recur(root)[1]
```

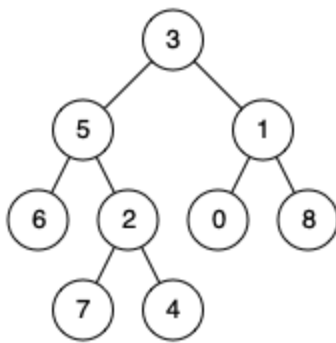
236 Lowest Common Ancestor of a Binary Tree (link)

Description

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.

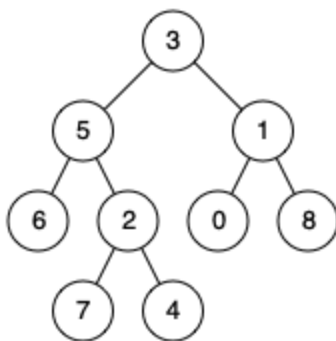
According to the [definition of LCA on Wikipedia](#): “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow **a node to be a descendant of itself**).”

Example 1:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.

Example 2:



Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself.

Example 3:

Input: root = [1,2], p = 1, q = 2

Output: 1**Constraints:**

- The number of nodes in the tree is in the range $[2, 10^5]$.
- $-10^9 \leq \text{Node.val} \leq 10^9$
- All `Node.val` are **unique**.
- $p \neq q$
- p and q will exist in the tree.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, x):
#         self.val = x
#         self.left = None
#         self.right = None

class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        def recur(root):
            if not root:
                return None
            if root == p or root == q:
                return root
            left = recur(root.left)
            right = recur(root.right)

            if left and right:
                return root
            elif not left or not right:
                return left if left else right
            else:
                return None

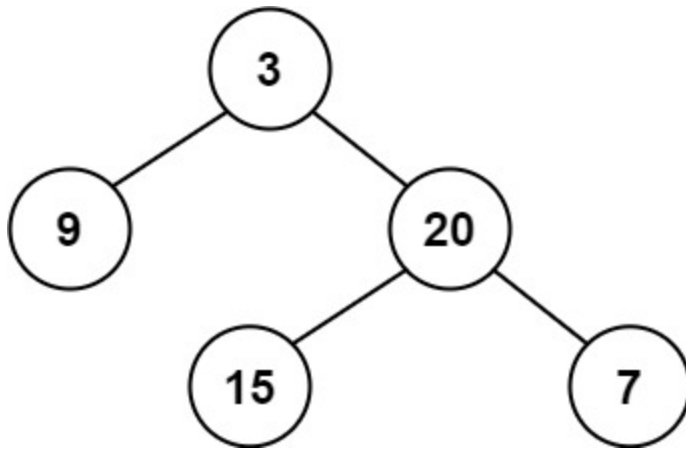
        return recur(root)
```

[110 Balanced Binary Tree \(link\)](#)

Description

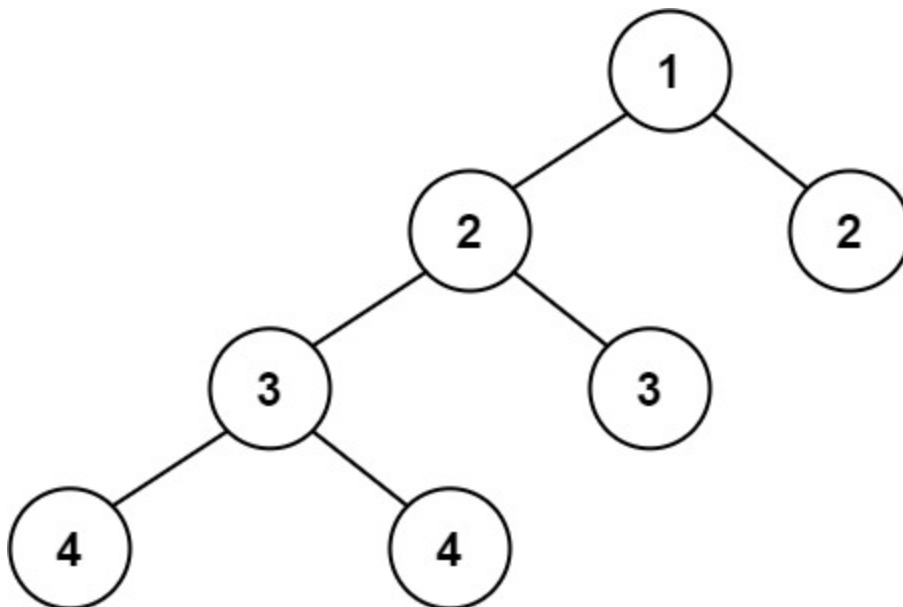
Given a binary tree, determine if it is **height-balanced**.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: true

Example 2:



Input: root = [1,2,2,3,3,null,null,4,4]
Output: false

Example 3:

Input: root = []
Output: true

Constraints:

- The number of nodes in the tree is in the range $[0, 5000]$.
- $-10^4 \leq \text{Node.val} \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def recur(root):
            if not root:
                return 0
            left = recur(root.left)
            if left == -1:
                return -1
            right = recur(root.right)
            if right == -1:
                return -1
            if abs(left - right) > 1:
                return -1
            return 1 + max(left, right)

        return recur(root) != -1
```

543 Diameter of Binary Tree (link)

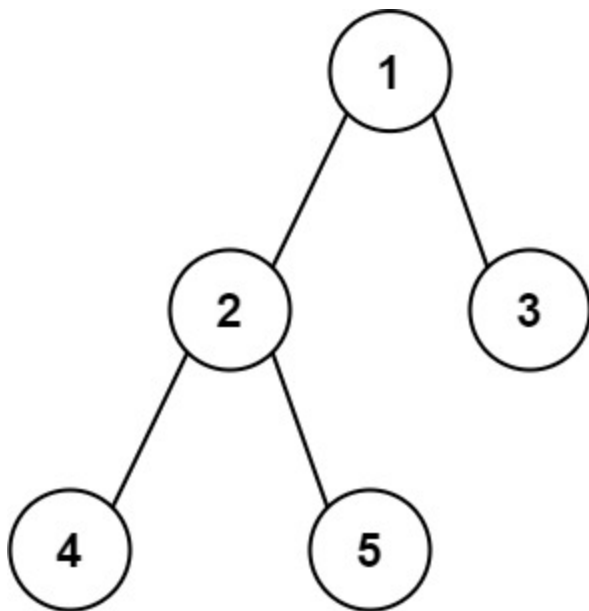
Description

Given the root of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range $[1, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        self.max_dia = 0

        def recur(root):
            if not root:
                return 0
            left = recur(root.left)
            right = recur(root.right)
            self.max_dia = max(self.max_dia, left + right)

            return 1 + max(left, right)
        recur(root)
        return self.max_dia
```

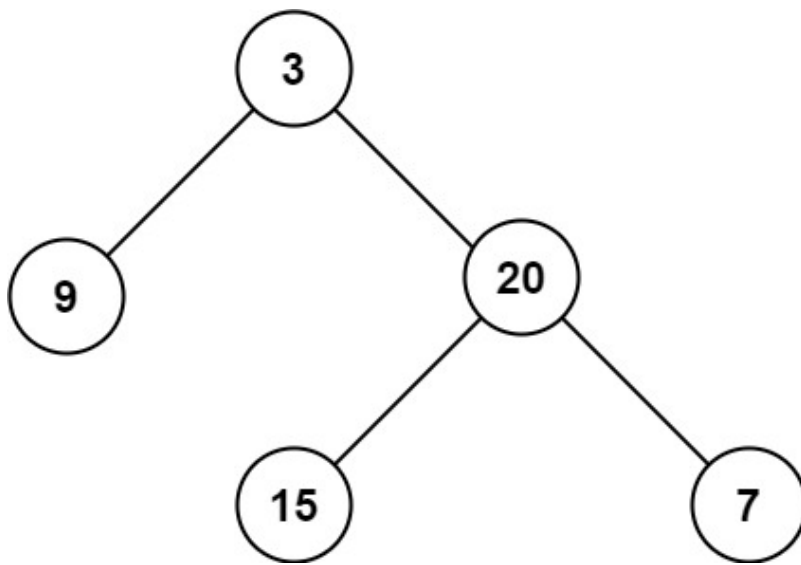
[104 Maximum Depth of Binary Tree \(link\)](#)

Description

Given the root of a binary tree, return *its maximum depth*.

A binary tree's **maximum depth** is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: 3

Example 2:

Input: root = [1,null,2]
Output: 2

Constraints:

- The number of nodes in the tree is in the range $[0, 10^4]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

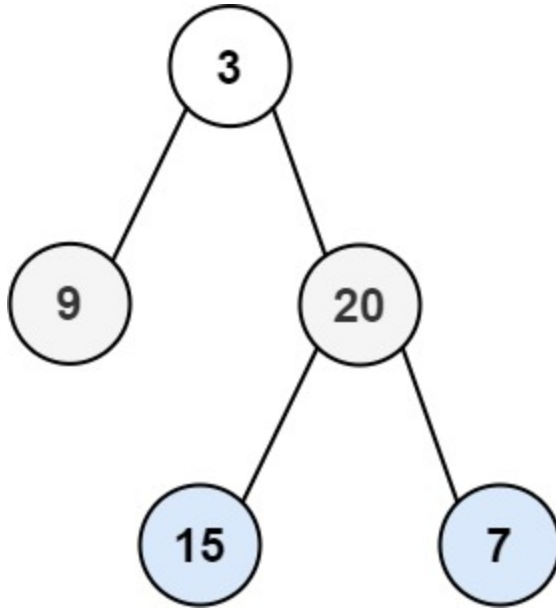
```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        left = self.maxDepth(root.left)
        right = self.maxDepth(root.right)
        return 1 + max(left, right)
```

[102 Binary Tree Level Order Traversal \(link\)](#)

Description

Given the root of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).

Example 1:



Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]

Example 2:

Input: root = [1]
Output: [[1]]

Example 3:

Input: root = []
Output: []

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- $-1000 \leq \text{Node.val} \leq 1000$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if not root:
            return []
        queue = deque()

        queue.append(root)
        output = []

        while queue:
            length = len(queue)
            curr = []

            for _ in range(length):
                node = queue.popleft()
                curr.append(node.val)
                if node.left:
                    queue.append(node.left)
                if node.right:
                    queue.append(node.right)
            output.append(curr)

        return output
```

662 Maximum Width of Binary Tree (link)

Description

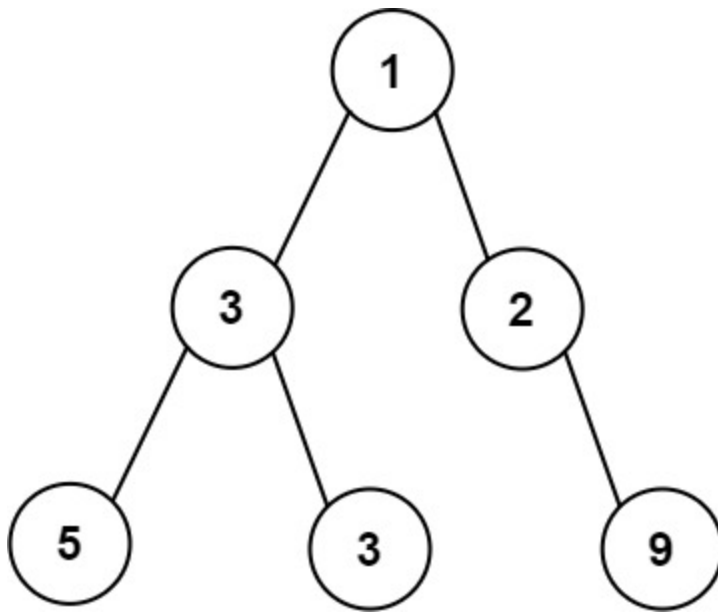
Given the root of a binary tree, return *the maximum width of the given tree*.

The **maximum width** of a tree is the maximum **width** among all levels.

The **width** of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.

It is **guaranteed** that the answer will in the range of a **32-bit** signed integer.

Example 1:

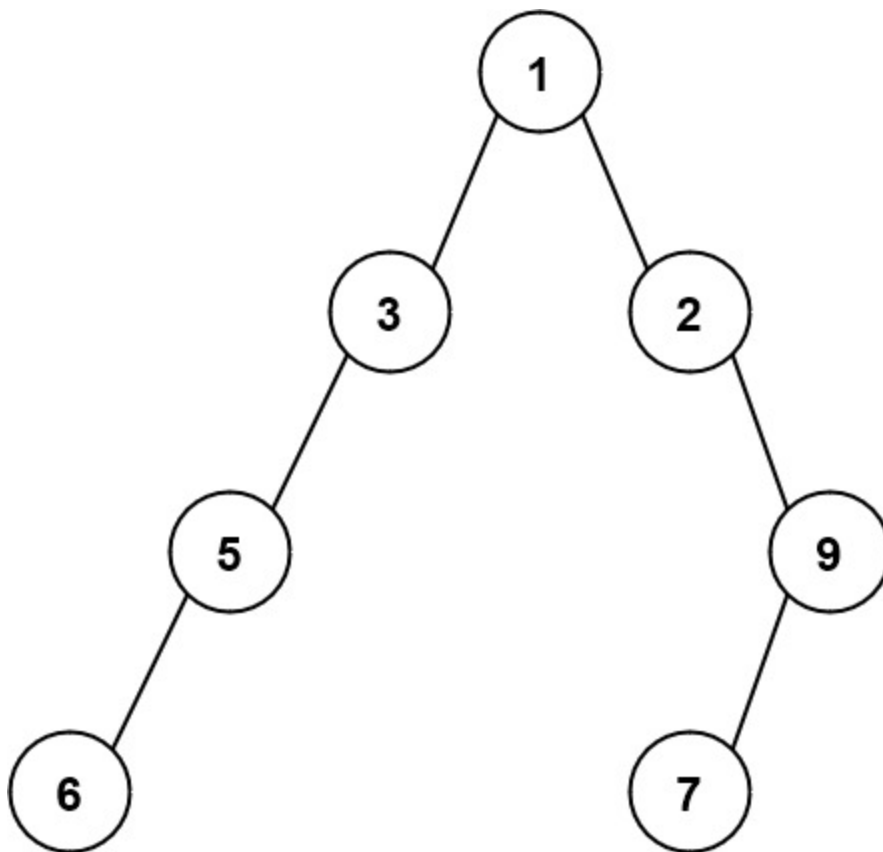


Input: root = [1,3,2,5,3,null,9]

Output: 4

Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).

Example 2:

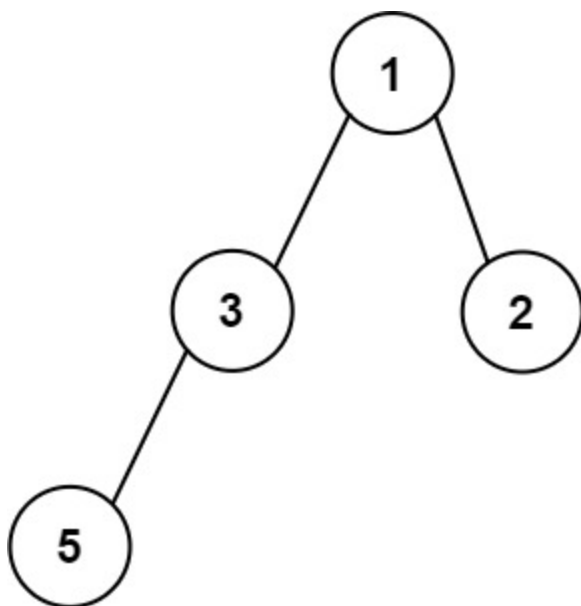


Input: root = [1,3,2,5,null,null,9,6,null,7]

Output: 7

Explanation: The maximum width exists in the fourth level with length 7 (6,null,n

Example 3:



Input: root = [1,3,2,5]

Output: 2

Explanation: The maximum width exists in the second level with length 2 (3,2).

Constraints:

- The number of nodes in the tree is in the range $[1, 3000]$.
- $-100 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def widthOfBinaryTree(self, root: Optional[TreeNode]) -> int:

        if not root:
            return 0
        max_width = 0
        queue = deque()

        queue.append((root, 0))

        while queue:
            length = len(queue)
            _, first_idx = queue[0]

            for _ in range(length):
                node, i = queue.popleft()
                curr_idx = i - first_idx
                print("current index" + str(curr_idx))
                print("first index" + str(first_idx))
                print("index " + str(i))
                if node.left:
                    queue.append((node.left, 2*curr_idx))
                if node.right:
                    queue.append((node.right, 2*curr_idx + 1))
            max_width = max(max_width, curr_idx + 1)

        return max_width
```

[94 Binary Tree Inorder Traversal \(link\)](#)

Description

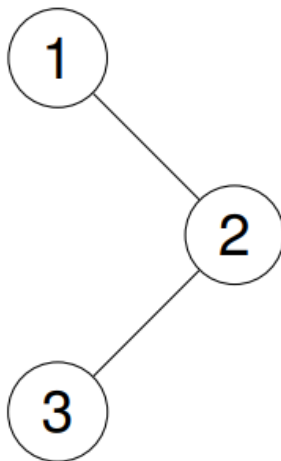
Given the root of a binary tree, return *the inorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,3,2]

Explanation:

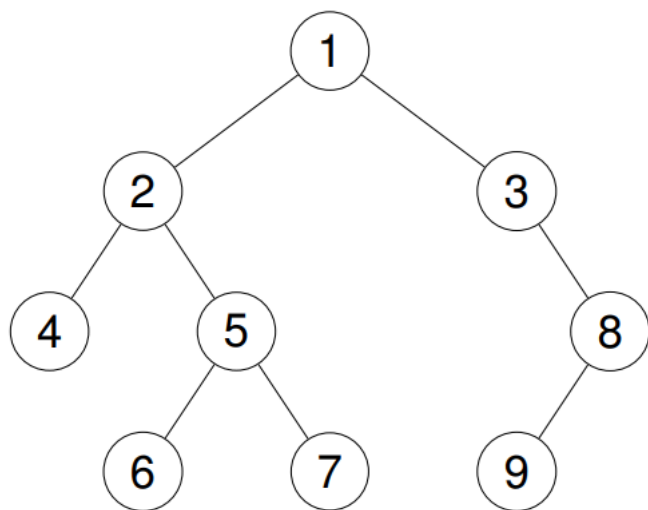


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,2,6,5,7,1,3,9,8]

Explanation:

**Example 3:****Input:** root = []**Output:** []**Example 4:****Input:** root = [1]**Output:** [1]**Constraints:**

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        node = root
        output = []

        while node:
            if not node.left:
                output.append(node.val)
                node = node.right
            else:
                pred = node.left
                while pred.right and pred.right != node:
                    pred = pred.right
                if not pred.right:
                    pred.right = node
                    node = node.left
                else:
                    pred.right = None
                    output.append(node.val)
                    node = node.right

        return output
```

[145 Binary Tree Postorder Traversal \(link\)](#)

Description

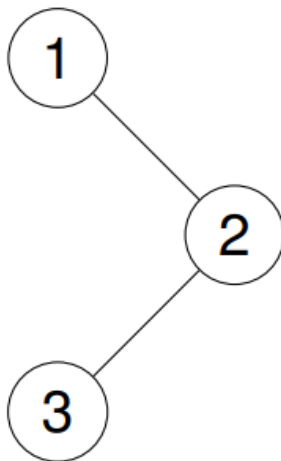
Given the root of a binary tree, return *the postorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [3,2,1]

Explanation:

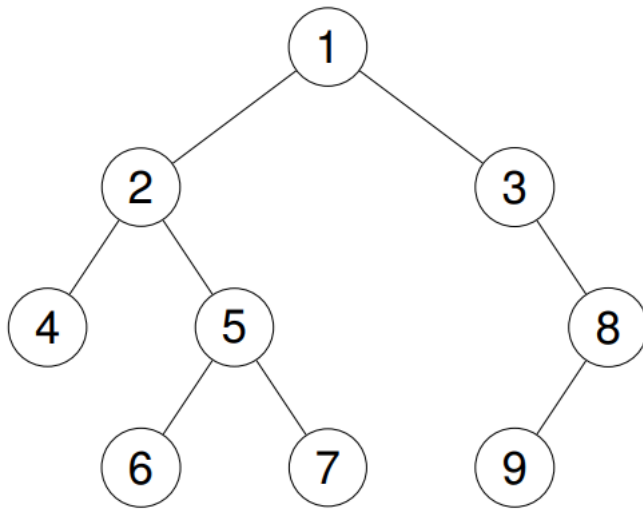


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [4,6,7,5,2,9,8,3,1]

Explanation:

**Example 3:**

Input: root = []

Output: []

Example 4:

Input: root = [1]

Output: [1]

Constraints:

- The number of the nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def postorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        output = []
        if not root:
            return output
        stack = [root]
        while stack:
            current = stack.pop()
            output.append(current.val)
            if current.left:
                stack.append(current.left)
            if current.right:
                stack.append(current.right)

        return output[::-1]
```

[144 Binary Tree Preorder Traversal \(link\)](#)

Description

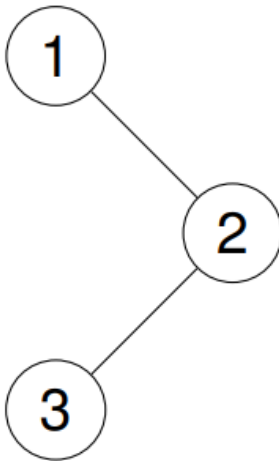
Given the root of a binary tree, return *the preorder traversal of its nodes' values*.

Example 1:

Input: root = [1,null,2,3]

Output: [1,2,3]

Explanation:

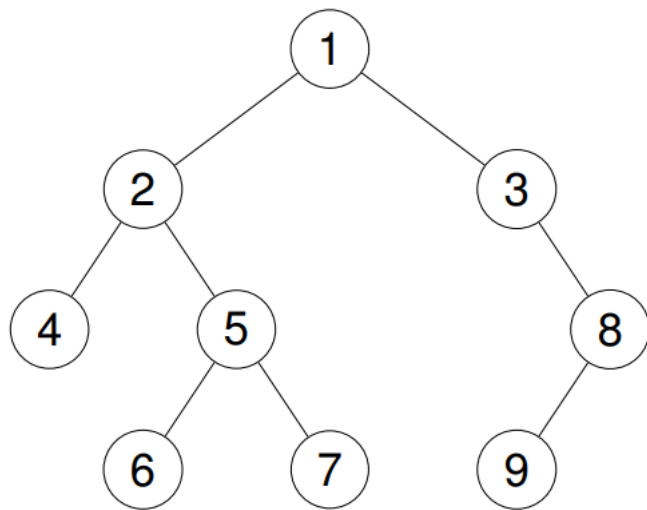


Example 2:

Input: root = [1,2,3,4,5,null,8,null,null,6,7,9]

Output: [1,2,4,5,6,7,3,8,9]

Explanation:

**Example 3:****Input:** root = []**Output:** []**Example 4:****Input:** root = [1]**Output:** [1]**Constraints:**

- The number of nodes in the tree is in the range $[0, 100]$.
- $-100 \leq \text{Node.val} \leq 100$

Follow up: Recursive solution is trivial, could you do it iteratively?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right
class Solution:
    def preorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        output = []
        stack = []
        if root:
            stack.append(root)

        while stack:
            current = stack.pop()
            output.append(current.val)
            if current.right:
                stack.append(current.right)
            if current.left:
                stack.append(current.left)

        return output
```

[686 Repeated String Match \(link\)](#)

Description

Given two strings *a* and *b*, return *the minimum number of times you should repeat string a so that string b is a substring of it*. If it is impossible for *b* to be a substring of *a* after repeating it, return *-1*.

Notice: string "abc" repeated 0 times is "", repeated 1 time is "abc" and repeated 2 times is "abcb".

Example 1:

Input: *a* = "abcd", *b* = "cdababcdab"

Output: 3

Explanation: We return 3 because by repeating *a* three times "ab**cdab**cdab", *b* is

Example 2:

Input: *a* = "a", *b* = "aa"

Output: 2

Constraints:

- $1 \leq a.length, b.length \leq 10^4$
- *a* and *b* consist of lowercase English letters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def repeatedStringMatch(self, a: str, b: str) -> int:

        repeated = a
        count = 1
        # Keep repeating until the repeated string is >= b
        while len(repeated) < len(b):
            repeated += a
            count += 1

        # Check current repeated string
        if b in repeated:
            return count
        # Check one more repeat to catch overlap at boundary
        repeated += a
        if b in repeated:
            return count + 1

        return -1
```

8 String to Integer (atoi) [\(link\)](#)

Description

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").
2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Example 1:

Input: `s = "42"`

Output: 42

Explanation:

The underlined characters are what is read in and the caret is the current reader
 Step 1: "42" (no characters read because there is no leading whitespace)

Step 2: "42" (no characters read because there is neither a '-' nor '+')

Step 3: "42" ("42" is read in)

Example 2:

Input: `s = "-042"`

Output: -42

Explanation:

Step 1: "___-042" (leading whitespace is read and ignored)

Step 2: " _-042" ('-' is read, so the result should be negative)

Step 3: " -042" ("042" is read in, leading zeros ignored in the result)

Example 3:

Input: s = "1337c0d3"

Output: 1337

Explanation:

Step 1: "1337c0d3" (no characters read because there is no leading whitespace)

Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')

Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)

Example 4:

Input: s = "0-1"

Output: 0

Explanation:

Step 1: "0-1" (no characters read because there is no leading whitespace)

Step 2: "0-1" (no characters read because there is neither a '-' nor '+')

Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)

Example 5:

Input: s = "words and 987"

Output: 0

Explanation:

Reading stops at the first non-digit character 'w'.

Constraints:

- $0 \leq s.length \leq 200$
- s consists of English letters (lower-case and upper-case), digits (0–9), ' ', '+', '-', and '.'.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def myAtoi(self, s: str) -> int:
        i = 0
        n = len(s)
        INT_MAX = 2**31 - 1
        INT_MIN = -2**31

        # Step 1: Ignore leading whitespaces
        while i < n and s[i] == ' ':
            i += 1

        # Step 2: Handle sign
        sign = 1
        if i < n and (s[i] == '+' or s[i] == '-'):
            if s[i] == '-':
                sign = -1
            i += 1

        # Step 3: Read digits and build the number
        num = 0
        while i < n and s[i].isdigit():
            num = num * 10 + int(s[i])
            i += 1

        # Step 4: Apply sign
        num *= sign

        # Step 5: Clamp within 32-bit signed integer range
        if num < INT_MIN:
            return INT_MIN
        if num > INT_MAX:
            return INT_MAX

        return num
```

[5 Longest Palindromic Substring \(link\)](#)

Description

Given a string s , return *the longest palindromic substring* in s .

Example 1:

Input: $s = \text{"babad"}$
Output: "bab"
Explanation: "aba" is also a valid answer.

Example 2:

Input: $s = \text{"cbbd"}$
Output: "bb"

Constraints:

- $1 \leq s.length \leq 1000$
- s consist of only digits and English letters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""

        palindrome = s[0]
        for i in range(len(s)):
            #odd
            l, r = i, i
            while l >= 0 and r < len(s) and s[l] == s[r]:
                if r - l + 1 > len(palindrome):
                    palindrome = s[l: r+1]
                l -= 1
                r += 1
            #even
            l, r = i, i+1
            while l >= 0 and r < len(s) and s[l] == s[r]:
                if r - l + 1 > len(palindrome):
                    palindrome = s[l: r+1]
                l -= 1
                r += 1

        return palindrome
```

[14 Longest Common Prefix \(link\)](#)

Description

Write a function to find the longest common prefix string amongst an array of strings.

If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs = ["flower","flow","flight"]  
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]  
Output: ""  
Explanation: There is no common prefix among the input strings.
```

Constraints:

- $1 \leq \text{strs.length} \leq 200$
- $0 \leq \text{strs}[i].\text{length} \leq 200$
- $\text{strs}[i]$ consists of only lowercase English letters if it is non-empty.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def longestCommonPrefix(self, strs: List[str]) -> str:
        prefix = strs[0]

        for word in strs[1:]:
            while not word.startswith(prefix):
                prefix = prefix[:-1]
                if prefix == "":
                    return ""
        return prefix
```

[13 Roman to Integer \(link\)](#)

Description

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- I can be placed before V (5) and X (10) to make 4 and 9.
- X can be placed before L (50) and C (100) to make 40 and 90.
- C can be placed before D (500) and M (1000) to make 400 and 900.

Given a roman numeral, convert it to an integer.

Example 1:

Input: s = "III"
Output: 3
Explanation: III = 3.

Example 2:

Input: s = "LVIII"
Output: 58
Explanation: L = 50, V= 5, III = 3.

Example 3:

Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.

Constraints:

- $1 \leq s.length \leq 15$
- s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').
- It is **guaranteed** that s is a valid roman numeral in the range [1, 3999].

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def romanToInt(self, s: str) -> int:
        dicts = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
        num = 0
        prev_val = 0
        for ch in reversed(s):
            current = dicts[ch]
            if current < prev_val:
                num -= current
            else:
                num += current
            prev_val = current

        return num
```

[151 Reverse Words in a String \(link\)](#)

Description

Given an input string *s*, reverse the order of the **words**.

A **word** is defined as a sequence of non-space characters. The **words** in *s* will be separated by at least one space.

Return *a string of the words in reverse order concatenated by a single space*.

Note that *s* may contain leading or trailing spaces or multiple spaces between two words. The returned string should only have a single space separating the words. Do not include any extra spaces.

Example 1:

Input: *s* = "the sky is blue"
Output: "blue is sky the"

Example 2:

Input: *s* = " hello world "
Output: "world hello"
Explanation: Your reversed string should not contain leading or trailing spaces.

Example 3:

Input: *s* = "a good example"
Output: "example good a"
Explanation: You need to reduce multiple spaces between two words to a single space.

Constraints:

- $1 \leq s.length \leq 10^4$
- *s* contains English letters (upper-case and lower-case), digits, and spaces ' '.
- There is **at least one** word in *s*.

Follow-up: If the string data type is mutable in your language, can you solve it **in-place** with $O(1)$ extra space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def reverseWords(self, s: str) -> str:
        s = s.split()
        return " ".join(reversed(s))
```

1036 Rotting Oranges (link)

Description

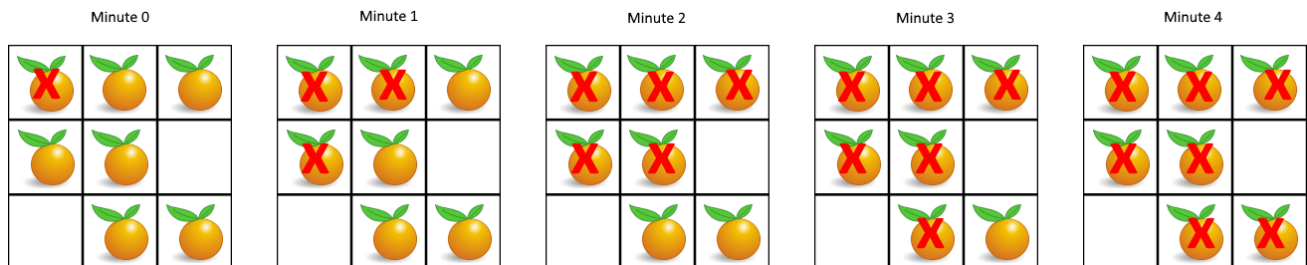
You are given an $m \times n$ grid where each cell can have one of three values:

- 0 representing an empty cell,
- 1 representing a fresh orange, or
- 2 representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:



Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4

Example 2:

Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotted.

Example 3:

Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is 0.

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $1 \leq m, n \leq 10$

- `grid[i][j]` is 0, 1, or 2.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def orangesRotting(self, grid: List[List[int]]) -> int:
        rotten = deque()
        fresh = 0
        rows = len(grid)
        cols = len(grid[0])
        for i in range(rows):
            for j in range(cols):
                if grid[i][j] == 2:
                    rotten.append((i,j))
                elif grid[i][j] == 1:
                    fresh += 1

        direction = [(-1,0), (1,0),(0,-1),(0,1)]
        time = 0
        while rotten and fresh:
            size = len(rotten)
            for _ in range(size):
                x, y = rotten.popleft()
                for dx, dy in direction:
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 1:
                        grid[nx][ny] = 2
                        fresh -= 1
                        rotten.append((nx, ny))

            time += 1

        if fresh == 0:
            return time
        return -1
```

[937 Online Stock Span \(link\)](#)

Description

Design an algorithm that collects daily price quotes for some stock and returns **the span** of that stock's price for the current day.

The **span** of the stock's price in one day is the maximum number of consecutive days (starting from that day and going backward) for which the stock price was less than or equal to the price of that day.

- For example, if the prices of the stock in the last four days is $[7, 2, 1, 2]$ and the price of the stock today is 2, then the span of today is 4 because starting from today, the price of the stock was less than or equal 2 for 4 consecutive days.
- Also, if the prices of the stock in the last four days is $[7, 34, 1, 2]$ and the price of the stock today is 8, then the span of today is 3 because starting from today, the price of the stock was less than or equal 8 for 3 consecutive days.

Implement the StockSpanner class:

- `StockSpanner()` Initializes the object of the class.
- `int next(int price)` Returns the **span** of the stock's price given that today's price is `price`.

Example 1:

Input

```
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]  
[[], [100], [80], [60], [70], [60], [75], [85]]
```

Output

```
[null, 1, 1, 1, 2, 1, 4, 6]
```

Explanation

```
StockSpanner stockSpanner = new StockSpanner();  
stockSpanner.next(100); // return 1  
stockSpanner.next(80);  // return 1  
stockSpanner.next(60);  // return 1  
stockSpanner.next(70);  // return 2  
stockSpanner.next(60);  // return 1  
stockSpanner.next(75);  // return 4, because the last 4 prices (including today's  
stockSpanner.next(85);  // return 6
```

Constraints:

- $1 \leq \text{price} \leq 10^5$
- At most 10^4 calls will be made to `next`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class StockSpanner:

    def __init__(self):
        self.stocks = []

    def next(self, price: int) -> int:
        span = 1
        while self.stocks and self.stocks[-1][0] <= price:
            span += self.stocks.pop()[1]

        self.stocks.append((price, span))

        return span

# Your StockSpanner object will be instantiated and called as such:
# obj = StockSpanner()
# param_1 = obj.next(price)
```

[155 Min Stack \(link\)](#)

Description

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the MinStack class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

Example 1:

Input

```
["MinStack","push","push","push","getMin","pop","top","getMin"]  
[[],[-2],[0],[-3],[],[],[],[ ]]
```

Output

```
[null,null,null,null,-3,null,0,-2]
```

Explanation

```
MinStack minStack = new MinStack();  
minStack.push(-2);  
minStack.push(0);  
minStack.push(-3);  
minStack.getMin(); // return -3  
minStack.pop();  
minStack.top();    // return 0  
minStack.getMin(); // return -2
```

Constraints:

- $-2^{31} \leq \text{val} \leq 2^{31} - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on **non-empty** stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class MinStack:

    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val: int) -> None:
        self.stack.append(val)
        # Push min of current value and last min
        if not self.min_stack:
            self.min_stack.append(val)
        else:
            self.min_stack.append(min(val, self.min_stack[-1]))

    def pop(self) -> None:
        self.stack.pop()
        self.min_stack.pop()

    def top(self) -> int:
        return self.stack[-1]

    def getMin(self) -> int:
        return self.min_stack[-1]
```

[225 Implement Stack using Queues \(link\)](#)

Description

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (push, top, pop, and empty).

Implement the MyStack class:

- `void push(int x)` Pushes element `x` to the top of the stack.
- `int pop()` Removes the element on the top of the stack and returns it.
- `int top()` Returns the element on the top of the stack.
- `boolean empty()` Returns `true` if the stack is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a queue, which means that only `push` to back, `peek/pop` from front, `size` and `is empty` operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Example 1:

Input

```
["MyStack", "push", "push", "top", "pop", "empty"]  
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 2, 2, false]
```

Explanation

```
MyStack myStack = new MyStack();  
myStack.push(1);  
myStack.push(2);  
myStack.top(); // return 2  
myStack.pop(); // return 2  
myStack.empty(); // return False
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to `push`, `pop`, `top`, and `empty`.
- All the calls to `pop` and `top` are valid.

Follow-up: Can you implement the stack using only one queue?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class MyStack:

    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()

    def push(self, x: int) -> None:
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1

    def pop(self) -> int:
        return self.q1.popleft()

    def top(self) -> int:
        return self.q1[0]

    def empty(self) -> bool:
        return not self.q1

# Your MyStack object will be instantiated and called as such:
# obj = MyStack()
# obj.push(x)
# param_2 = obj.pop()
# param_3 = obj.top()
# param_4 = obj.empty()
```

[232 Implement Queue using Stacks \(link\)](#)

Description

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- `void push(int x)` Pushes element `x` to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes:

- You must use **only** standard operations of a stack, which means only `push` to `top`, `peek/pop` from `top`, `size`, and `is empty` operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

Input

```
["MyQueue", "push", "push", "peek", "pop", "empty"]  
[[], [1], [2], [], [], []]
```

Output

```
[null, null, null, 1, 1, false]
```

Explanation

```
MyQueue myQueue = new MyQueue();  
myQueue.push(1); // queue is: [1]  
myQueue.push(2); // queue is: [1, 2] (leftmost is front of the queue)  
myQueue.peek(); // return 1  
myQueue.pop(); // return 1, queue is [2]  
myQueue.empty(); // return false
```

Constraints:

- $1 \leq x \leq 9$
- At most 100 calls will be made to `push`, `pop`, `peek`, and `empty`.
- All the calls to `pop` and `peek` are valid.

Follow-up: Can you implement the queue such that each operation is [amortized](#) $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class MyQueue:

    def __init__(self):
        self.stack1 = [] # Input stack
        self.stack2 = [] # Output stack

    def push(self, x: int) -> None:
        self.stack1.append(x)

    def pop(self) -> int:
        self._move_stack1_to_stack2()
        return self.stack2.pop()

    def peek(self) -> int:
        self._move_stack1_to_stack2()
        return self.stack2[-1]

    def empty(self) -> bool:
        return not self.stack1 and not self.stack2

    def _move_stack1_to_stack2(self):
        if not self.stack2: # Only move if stack2 is empty
            while self.stack1:
                self.stack2.append(self.stack1.pop())
```

[20 Valid Parentheses \(link\)](#)

Description

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "("

Output: false

Example 4:

Input: *s* = "([])"

Output: true

Example 5:

Input: *s* = "([)]"

Output: false

Constraints:

- $1 \leq s.length \leq 10^4$
- *s* consists of parentheses only '()[]{}'.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def isValid(self, s: str) -> bool:
        stack = []
        for l in s:
            if l in ['(', '{', '[']:
                stack.append(l)
            elif stack and stack[-1] == '(' and l == ')' :
                stack.pop(-1)
            elif stack and stack[-1] == '{' and l == '}':
                stack.pop(-1)
            elif stack and stack[-1] == '[' and l == ']':
                stack.pop(-1)
            else:
                return False
        if stack:
            return False
        return True
```

[496 Next Greater Element I \(link\)](#)

Description

The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each $0 \leq i < \text{nums1.length}$, find the index j such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the **next greater element** as described above.

Example 1:

Input: `nums1 = [4,1,2]`, `nums2 = [1,3,4,2]`

Output: `[-1,3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 4 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the
- 1 is underlined in `nums2 = [1,3,4,2]`. The next greater element is 3.
- 2 is underlined in `nums2 = [1,3,4,2]`. There is no next greater element, so the

Example 2:

Input: `nums1 = [2,4]`, `nums2 = [1,2,3,4]`

Output: `[3,-1]`

Explanation: The next greater element for each value of `nums1` is as follows:

- 2 is underlined in `nums2 = [1,2,3,4]`. The next greater element is 3.
- 4 is underlined in `nums2 = [1,2,3,4]`. There is no next greater element, so the

Constraints:

- $1 \leq \text{nums1.length} \leq \text{nums2.length} \leq 1000$
- $0 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^4$
- All integers in `nums1` and `nums2` are **unique**.
- All the integers of `nums1` also appear in `nums2`.

Follow up: Could you find an $O(\text{nums1.length} + \text{nums2.length})$ solution?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def nextGreaterElement(self, nums1: List[int], nums2: List[int]) -> List[int]:
        stack = []
        hmap = {}

        for num in reversed(nums2):
            while stack and stack[-1] <= num:
                stack.pop(-1)
            hmap[num] = stack[-1] if stack else -1
            stack.append(num)
        result = []
        for num in nums1:
            result.append(hmap[num])
        return result
```

[4 Median of Two Sorted Arrays \(link\)](#)

Description

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return **the median** of the two sorted arrays.

The overall run time complexity should be $O(\log(m+n))$.

Example 1:

Input: `nums1 = [1,3]`, `nums2 = [2]`

Output: 2.00000

Explanation: merged array = `[1,2,3]` and median is 2.

Example 2:

Input: `nums1 = [1,2]`, `nums2 = [3,4]`

Output: 2.50000

Explanation: merged array = `[1,2,3,4]` and median is $(2 + 3) / 2 = 2.5$.

Constraints:

- `nums1.length == m`
- `nums2.length == n`
- $0 \leq m \leq 1000$
- $0 \leq n \leq 1000$
- $1 \leq m + n \leq 2000$
- $-10^6 \leq \text{nums1}[i], \text{nums2}[i] \leq 10^6$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def findMedianSortedArrays(self, nums1: List[int], nums2: List[int]) -> float:
        # ✅ Step 1: Ensure nums1 is the smaller array
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1

        # ✅ Step 2: Set binary search bounds on the smaller array
        n1, n2 = len(nums1), len(nums2)
        start, end = 0, n1

        while start <= end:
            # ✅ Step 3: Partition both arrays
            cut1 = (start + end) // 2
            cut2 = (n1 + n2 + 1) // 2 - cut1

            # left elements (just before the cuts)
            l1 = float('-inf') if cut1 == 0 else nums1[cut1 - 1]
            l2 = float('-inf') if cut2 == 0 else nums2[cut2 - 1]

            # right elements (just after the cuts)
            r1 = float('inf') if cut1 == len(nums1) else nums1[cut1]
            r2 = float('inf') if cut2 == len(nums2) else nums2[cut2]

            # ✅ Step 5: Check if partition is valid
            if l1 <= r2 and l2 <= r1: # l1 <= r2 and l2 <= r1
                # ✅ Step 6: Return median based on total length (odd/even)
                if (n1 + n2) % 2 == 0:
                    return (max(l1, l2) + min(r1, r2)) / 2
                else:
                    return max(l1, l2)
            elif l1 > r2:
                # Move left in nums1
                end = cut1 - 1
            else:
                # Move right in nums1
                start = cut1 + 1

        return 0.0 # placeholder
```

[33 Search in Rotated Sorted Array \(link\)](#)

Description

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly rotated** at an unknown pivot index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be rotated at pivot index 3 and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Example 1:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`
Output: 4

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`, `target = 3`
Output: -1

Example 3:

Input: `nums = [1]`, `target = 0`
Output: -1

Constraints:

- $1 \leq \text{nums.length} \leq 5000$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- All values of `nums` are **unique**.
- `nums` is an ascending array that is possibly rotated.
- $-10^4 \leq \text{target} \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def search(self, nums: List[int], target: int) -> int:
        start = 0
        end = len(nums) - 1

        while start <= end:
            mid = (start + end) // 2
            if nums[mid] == target:
                return mid

            if nums[start] <= nums[mid]:
                if nums[start] <= target < nums[mid]:
                    end = mid
                else:
                    start = mid + 1

            else:
                if nums[mid] < target <= nums[end]:
                    start = mid + 1
                else:
                    end = mid

        return -1
```

[540 Single Element in a Sorted Array \(link\)](#)

Description

You are given a sorted array consisting of only integers where every element appears exactly twice, except for one element which appears exactly once.

Return *the single element that appears only once*.

Your solution must run in $O(\log n)$ time and $O(1)$ space.

Example 1:

Input: `nums = [1,1,2,3,3,4,4,8,8]`
Output: `2`

Example 2:

Input: `nums = [3,3,7,7,10,11,11]`
Output: `10`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $0 \leq \text{nums}[i] \leq 10^5$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def singleNonDuplicate(self, nums):
        def recur(start, end):
            # ✅ Base case: only one element left
            if start == end:
                return nums[start]

            mid = (start + end) // 2

            # Make sure mid is even for pairing
            if mid % 2 == 1:
                mid -= 1

            # ✅ Correct pair found → single element is on the right
            if nums[mid] == nums[mid + 1]:
                return recur(mid + 2, end)
            else:
                return recur(start, mid)

        return recur(0, len(nums) - 1)
```

46 Permutations (link)

Description

Given an array `nums` of distinct integers, return all the possible permutations. You can return the answer in **any order**.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- $1 \leq \text{nums.length} \leq 6$
- $-10 \leq \text{nums}[i] \leq 10$
- All the integers of `nums` are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def permute(self, nums: List[int]) -> List[List[int]]:
        result = []

        def recur(ind):
            if ind == len(nums):
                result.append(nums[:])
                return
            for i in range(ind, len(nums)):
                nums[i], nums[ind] = nums[ind], nums[i]
                recur(ind + 1)
                nums[i], nums[ind] = nums[ind], nums[i]

        recur(0)
        return result
```

[131 Palindrome Partitioning \(link\)](#)

Description

Given a string s , partition s such that every substring of the partition is a **palindrome**. Return *all possible palindrome partitioning of s* .

Example 1:

```
Input: s = "aab"  
Output: [["a","a","b"],["aa","b"]]
```

Example 2:

```
Input: s = "a"  
Output: [["a"]]
```

Constraints:

- $1 \leq s.length \leq 16$
- s contains only lowercase English letters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def partition(self, s: str) -> List[List[str]]:
        result = []
        ds = []

        def is_palindrome(sub):
            return sub == sub[::-1]

        def recur(ind):
            if ind == len(s):
                result.append(ds[:])
                return
            for i in range(ind, len(s)):
                sub = s[ind : i+1]
                if is_palindrome(sub):
                    ds.append(sub)
                    recur(i+1)
                    ds.pop()

        recur(0)
        return result
```

[40 Combination Sum II \(link\)](#)

Description

Given a collection of candidate numbers (`candidates`) and a target number (`target`), find all unique combinations in `candidates` where the candidate numbers sum to `target`.

Each number in `candidates` may only be used **once** in the combination.

Note: The solution set must not contain duplicate combinations.

Example 1:

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
Output:
[
  [1,1,6],
  [1,2,5],
  [1,7],
  [2,6]
]
```

Example 2:

```
Input: candidates = [2,5,2,1,2], target = 5
Output:
[
  [1,2,2],
  [5]
]
```

Constraints:

- $1 \leq \text{candidates.length} \leq 100$
- $1 \leq \text{candidates}[i] \leq 50$
- $1 \leq \text{target} \leq 30$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def combinationSum2(self, candidates: List[int], target: int) -> List[List[int]]:
        result = []
        ds = []
        candidates.sort()

        def recur(ind, tgt):
            if ind == len(candidates):
                if tgt == 0:
                    result.append(ds[:])
                return
            if candidates[ind] <= tgt:
                ds.append(candidates[ind])
                recur(ind + 1, tgt - candidates[ind])
                ds.pop()

            while ind + 1 < len(candidates) and candidates[ind] == candidates[ind + 1]:
                ind += 1
            recur(ind + 1, tgt)

        recur(0, target)
        return result
```

[39 Combination Sum \(link\)](#)

Description

Given an array of **distinct** integers `candidates` and a target integer `target`, return *a list of all **unique combinations** of candidates where the chosen numbers sum to target*. You may return the combinations in **any order**.

The **same** number may be chosen from `candidates` an **unlimited number of times**. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

Input: `candidates = [2,3,6,7], target = 7`

Output: `[[2,2,3],[7]]`

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times
7 is a candidate, and $7 = 7$.

These are the only two combinations.

Example 2:

Input: `candidates = [2,3,5], target = 8`

Output: `[[2,2,2,2],[2,3,3],[3,5]]`

Example 3:

Input: `candidates = [2], target = 1`

Output: `[]`

Constraints:

- $1 \leq \text{candidates.length} \leq 30$
- $2 \leq \text{candidates}[i] \leq 40$
- All elements of `candidates` are **distinct**.
- $1 \leq \text{target} \leq 40$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def combinationSum(self, candidates: List[int], target: int) -> List[List[int]]:
        result = []
        ds = []

        def recur(ind, ds, tgt):
            if ind == len(candidates):
                if tgt == 0:
                    result.append(ds[:])
                return
            if candidates[ind] <= tgt:
                ds.append(candidates[ind])
                recur(ind, ds, tgt - candidates[ind])
                ds.pop()
            recur(ind + 1, ds, tgt)

        recur(0, ds, target)
        return result
```

[90 Subsets II \(link\)](#)

Description

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: `nums = [1,2,2]`
Output: `[[], [1], [1,2], [1,2,2], [2], [2,2]]`

Example 2:

Input: `nums = [0]`
Output: `[[], [0]]`

Constraints:

- `1 <= nums.length <= 10`
- `-10 <= nums[i] <= 10`

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def subsetsWithDup(self, nums: List[int]) -> List[List[int]]:
        result = []
        nums.sort()

        def recur(ind, ds):
            result.append(ds[:])
            for i in range(ind, len(nums)):
                if i > ind and nums[i] == nums[i-1]:
                    continue
                ds.append(nums[i])
                recur(i + 1, ds)
                ds.pop()
        recur(0, [])
        return result
```

[78 Subsets \(link\)](#)

Description

Given an integer array `nums` of **unique** elements, return *all possible subsets (the power set)*.

The solution set **must not** contain duplicate subsets. Return the solution in **any order**.

Example 1:

Input: `nums = [1,2,3]`

Output: `[[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]`

Example 2:

Input: `nums = [0]`

Output: `[[], [0]]`

Constraints:

- $1 \leq \text{nums.length} \leq 10$
- $-10 \leq \text{nums}[i] \leq 10$
- All the numbers of `nums` are **unique**.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def subsets(self, nums: List[int]) -> List[List[int]]:
        ssets = [[]]
        for num in nums:
            new_ssets = []
            for sets in ssets:
                new_ssets.append(sets + [num])
            ssets.extend(new_ssets)
        return ssets
```

21 Merge Two Sorted Lists (link)

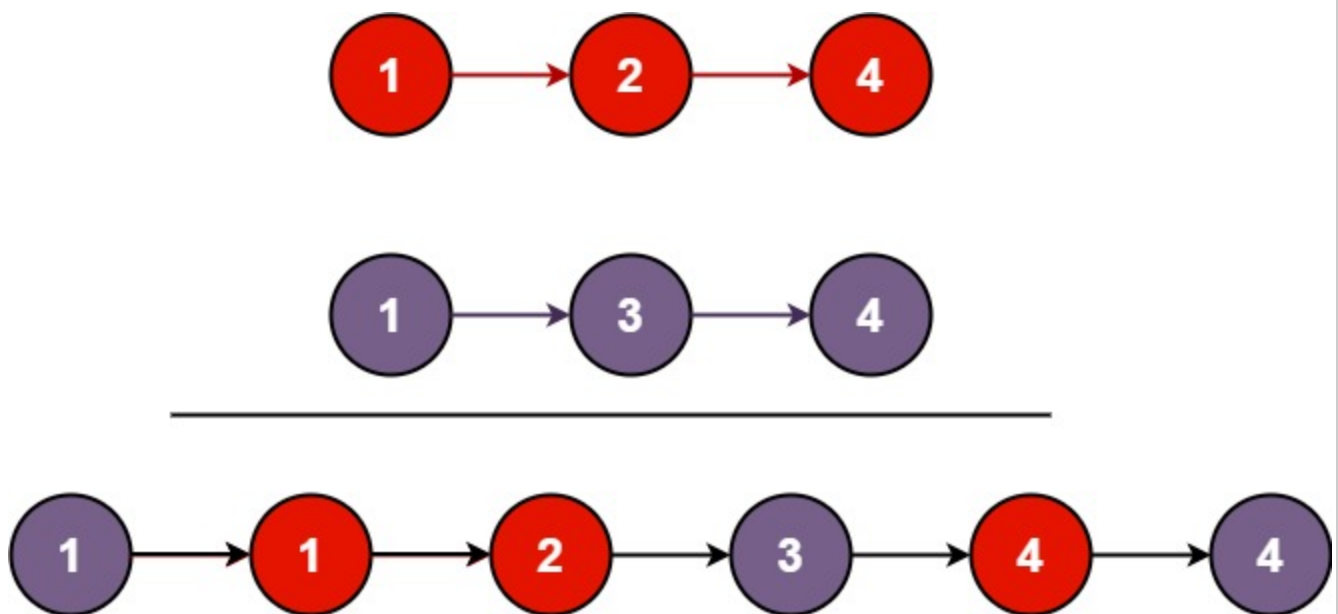
Description

You are given the heads of two sorted linked lists `list1` and `list2`.

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.

Return *the head of the merged linked list*.

Example 1:



Input: `list1 = [1,2,4]`, `list2 = [1,3,4]`
Output: `[1,1,2,3,4,4]`

Example 2:

Input: `list1 = []`, `list2 = []`
Output: `[]`

Example 3:

Input: `list1 = []`, `list2 = [0]`
Output: `[0]`

Constraints:

- The number of nodes in both lists is in the range `[0, 50]`.

- `-100 <= Node.val <= 100`
- Both `list1` and `list2` are sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def mergeTwoLists(self, list1: Optional[ListNode], list2: Optional[ListNode])
        current = ListNode()
        dummy = current
        while list1 and list2:
            if list1.val < list2.val:
                current.next = list1
                list1 = list1.next
            else:
                current.next = list2
                list2 = list2.next
            current = current.next

        current.next = list1 if list1 else list2

        return dummy.next
```

237 Delete Node in a Linked List (link)

Description

There is a singly-linked list head and we want to delete a node node in it.

You are given the node to be deleted node. You will **not be given access** to the first node of head.

All the values of the linked list are **unique**, and it is guaranteed that the given node node is not the last node in the linked list.

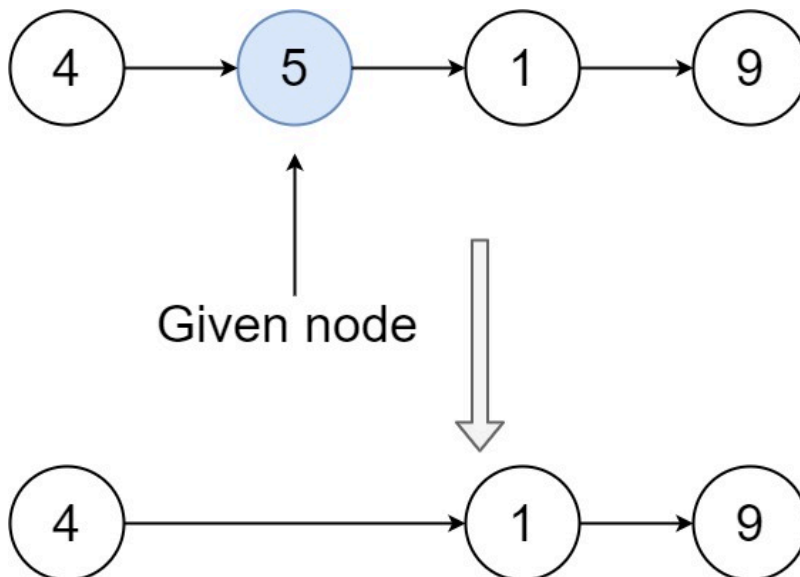
Delete the given node. Note that by deleting the node, we do not mean removing it from memory. We mean:

- The value of the given node should not exist in the linked list.
- The number of nodes in the linked list should decrease by one.
- All the values before node should be in the same order.
- All the values after node should be in the same order.

Custom testing:

- For the input, you should provide the entire linked list head and the node to be given node. node should not be the last node of the list and should be an actual node in the list.
- We will build the linked list and pass the node to your function.
- The output will be the entire list after calling your function.

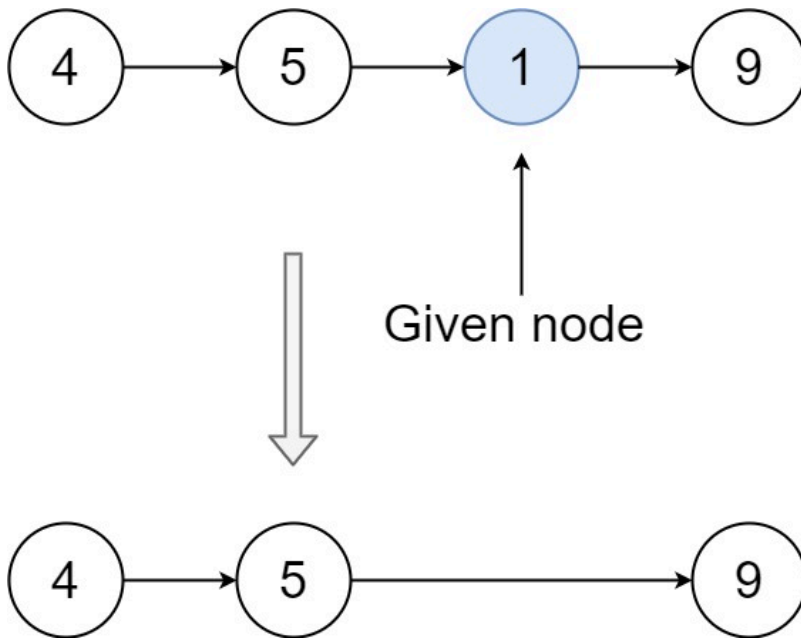
Example 1:



Input: head = [4,5,1,9], node = 5

Output: [4,1,9]

Explanation: You are given the second node with value 5, the linked list should be

Example 2:

Input: head = [4,5,1,9], node = 1

Output: [4,5,9]

Explanation: You are given the third node with value 1, the linked list should be

Constraints:

- The number of the nodes in the given list is in the range [2, 1000].
- $-1000 \leq \text{Node.val} \leq 1000$
- The value of each node in the list is **unique**.
- The node to be deleted is **in the list** and is **not a tail** node.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, x):
#         self.val = x
#         self.next = None

class Solution:
    def deleteNode(self, node):
        """
        :type node: ListNode
        :rtype: void Do not return anything, modify node in-place instead.
        """
        node.val = node.next.val
        node.next = node.next.next
```

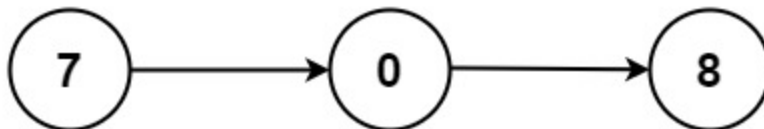
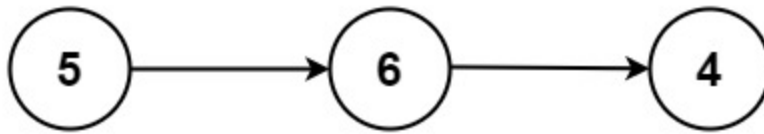
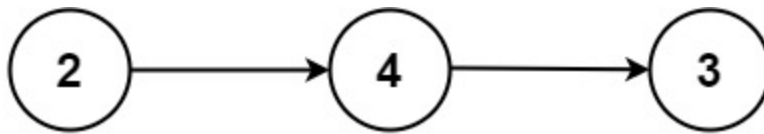
[2 Add Two Numbers \(link\)](#)

Description

You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:



Input: l1 = [2,4,3], l2 = [5,6,4]

Output: [7,0,8]

Explanation: 342 + 465 = 807.

Example 2:

Input: l1 = [0], l2 = [0]

Output: [0]

Example 3:

Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]

Output: [8,9,9,9,0,0,0,1]

Constraints:

- The number of nodes in each linked list is in the range $[1, 100]$.
- $0 \leq \text{Node.val} \leq 9$
- It is guaranteed that the list represents a number that does not have leading zeros.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def addTwoNumbers(self, l1: Optional[ListNode], l2: Optional[ListNode]) -> Optional[ListNode]:
        current = output = ListNode()
        carry = 0

        while l1 and l2:
            sum = l1.val + l2.val + carry
            if sum > 9:
                carry = 1
            else:
                carry = 0
            node = ListNode(sum%10)
            node.next = None
            current.next = node
            current = current.next
            l1 = l1.next
            l2 = l2.next
            print(node.val)

        if l1:
            while l1:
                sum = l1.val + carry
                if sum > 9:
                    carry = 1
                else:
                    carry = 0
                node = ListNode(sum%10)
                node.next = None
                current.next = node
                current = current.next
                l1 = l1.next
                print(node.val)

        if l2:
            while l2:
                sum = l2.val + carry
                if sum > 9:
                    carry = 1
                else:
                    carry = 0
                node = ListNode(sum%10)
                node.next = None
                current.next = node
                current = current.next
                l2 = l2.next
                print(node.val)

        if carry:
            node = ListNode(1)
            current.next = node
            current = current.next
            print(node.val)
```

```
return output.next
```

26 Remove Duplicates from Sorted Array [\(link\)](#)

Description

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**. Then return *the number of unique elements in* `nums`.

Consider the number of unique elements of `nums` to be `k`, to get accepted, you need to do the following things:

- Change the array `nums` such that the first `k` elements of `nums` contain the unique elements in the order they were present in `nums` initially. The remaining elements of `nums` are not important as well as the size of `nums`.
- Return `k`.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,2]`

Output: `2, nums = [1,2,_]`

Explanation: Your function should return `k = 2`, with the first two elements of `nums` being `1` and `2`. It does not matter what you leave beyond the returned `k` (hence they are underscored).

Example 2:

Input: `nums = [0,0,1,1,1,2,2,3,3,4]`

Output: `5, nums = [0,1,2,3,4,_,_,_,_,_]`

Explanation: Your function should return `k = 5`, with the first five elements of `nums` being `0`, `1`, `2`, `3`, and `4`. It does not matter what you leave beyond the returned `k` (hence they are underscored).

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-100 \leq \text{nums}[i] \leq 100$
- `nums` is sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        if not nums:
            return 0

        i = 0 # slow pointer
        for j in range(1, len(nums)): # fast pointer
            if nums[j] != nums[i]:
                i += 1
                nums[i] = nums[j]

        return i + 1
```

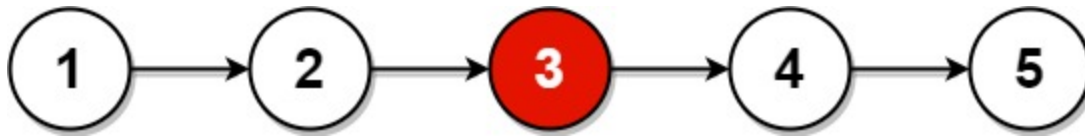
908 Middle of the Linked List (link)

Description

Given the head of a singly linked list, return *the middle node of the linked list*.

If there are two middle nodes, return **the second middle** node.

Example 1:

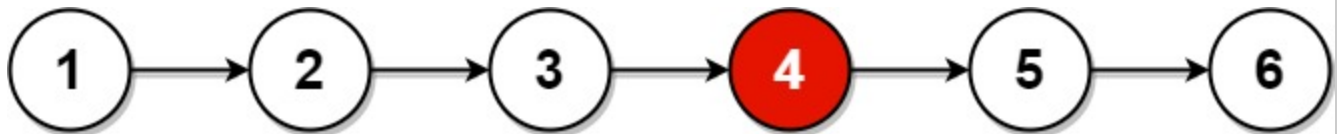


Input: head = [1,2,3,4,5]

Output: [3,4,5]

Explanation: The middle node of the list is node 3.

Example 2:



Input: head = [1,2,3,4,5,6]

Output: [4,5,6]

Explanation: Since the list has two middle nodes with values 3 and 4, we return the second middle node, which is 4.

Constraints:

- The number of nodes in the list is in the range [1, 100].
- $1 \leq \text{Node.val} \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def middleNode(self, head: Optional[ListNode]) -> Optional[ListNode]:
        slow = fast = head

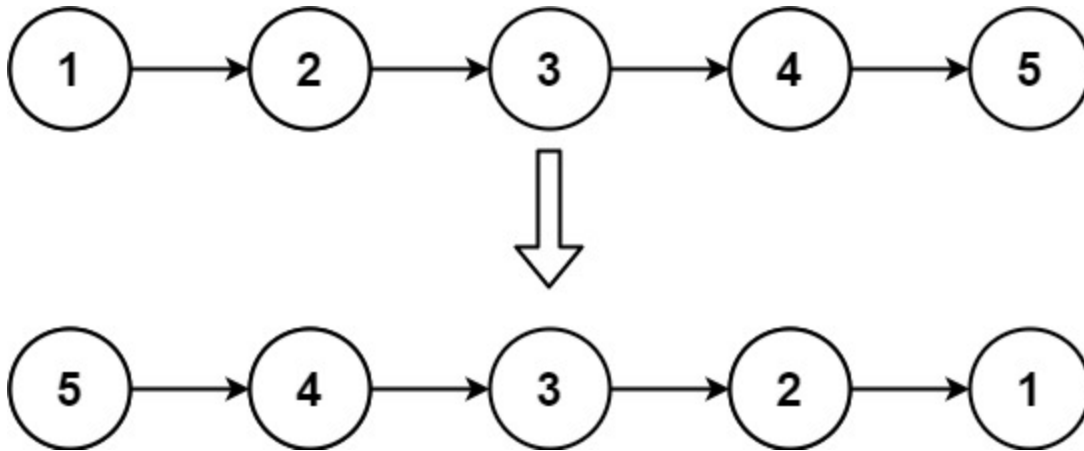
        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
        return slow
```

206 Reverse Linked List (link)

Description

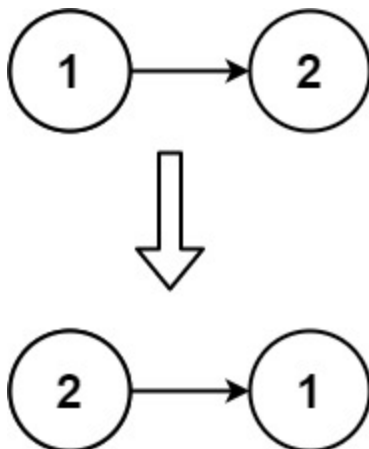
Given the head of a singly linked list, reverse the list, and return *the reversed list*.

Example 1:



Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]
Output: [2,1]

Example 3:

Input: head = []
Output: []

Constraints:

- The number of nodes in the list is the range $[0, 5000]$.
- $-5000 \leq \text{Node.val} \leq 5000$

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def recursive(self, current, prev):
        if current == None:
            return prev
        temp = current.next
        current.next = prev
        prev = current
        return self.recursive(temp, prev)

    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev = None
        current = head
        return self.recursive(current, prev)
```

[169 Majority Element \(link\)](#)

Description

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`
Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`
Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-109 <= nums[i] <= 109`

Follow-up: Could you solve the problem in linear time and in $O(1)$ space?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def majorityElement(self, nums: List[int]) -> int:
        num = nums[0]
        count = 1
        for i in range(1, len(nums)):
            if nums[i] == num:
                count += 1
            else:
                count -= 1
            if count == 0:
                num = nums[i]
                count += 1
        print(count)

    return num
```

[121 Best Time to Buy and Sell Stock \(link\)](#)

Description

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return *the maximum profit you can achieve from this transaction*. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5

Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2:

Input: `prices = [7,6,4,3,1]`

Output: 0

Explanation: In this case, no transactions are done and the max profit = 0.

Constraints:

- $1 \leq \text{prices.length} \leq 10^5$
- $0 \leq \text{prices}[i] \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        minn = prices[0]
        maxx = 0

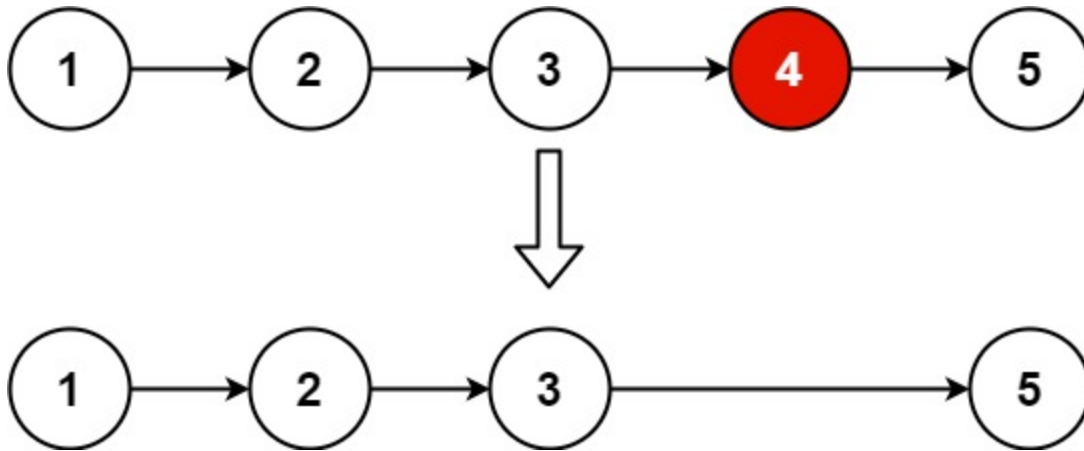
        for i in range(len(prices)):
            if prices[i] < minn:
                minn = prices[i]
            elif(maxx < prices[i] - minn):
                maxx = prices[i] - minn
        return maxx
```

19 Remove Nth Node From End of List (link)

Description

Given the head of a linked list, remove the n^{th} node from the end of the list and return its head.

Example 1:



Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]

Example 2:

Input: head = [1], n = 1
Output: []

Example 3:

Input: head = [1,2], n = 1
Output: [1]

Constraints:

- The number of nodes in the list is sz.
- $1 \leq sz \leq 30$
- $0 \leq \text{Node.val} \leq 100$
- $1 \leq n \leq sz$

Follow up: Could you do this in one pass?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
# Definition for singly-linked list.
# class ListNode:
#     def __init__(self, val=0, next=None):
#         self.val = val
#         self.next = next
class Solution:
    def removeNthFromEnd(self, head: Optional[ListNode], n: int) -> Optional[ListNode]:
        dummy = ListNode(0)
        dummy.next = head

        # Initialize two pointers
        first = dummy
        second = dummy

        # Move first pointer n+1 steps ahead
        for i in range(n + 1):
            first = first.next

        # Move both pointers until first reaches the end
        while first:
            first = first.next
            second = second.next

        # Remove the nth node from the end
        second.next = second.next.next

        return dummy.next
```

[3 Longest Substring Without Repeating Characters \(link\)](#)

Description

Given a string s , find the length of the **longest substring** without duplicate characters.

Example 1:

Input: $s = \text{"abcabcbb"}$

Output: 3

Explanation: The answer is "abc", with the length of 3.

Example 2:

Input: $s = \text{"bbbbbb"}$

Output: 1

Explanation: The answer is "b", with the length of 1.

Example 3:

Input: $s = \text{"pwwkew"}$

Output: 3

Explanation: The answer is "wke", with the length of 3.

Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.

Constraints:

- $0 \leq s.length \leq 5 * 10^4$
- s consists of English letters, digits, symbols and spaces.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        if not s:
            return 0
        set_str = set()
        max_cnt = 0
        l = r = 0
        while(l <= r and r <= len(s)-1):
            if s[r] not in set_str:
                set_str.add(s[r])
                max_cnt = max(max_cnt, r-l + 1)
                r += 1
            else:
                set_str.remove(s[l])
                l += 1

        return max_cnt
```

62 Unique Paths (link)

Description

There is a robot on an $m \times n$ grid. The robot is initially located at the **top-left corner** (i.e., `grid[0][0]`). The robot tries to move to the **bottom-right corner** (i.e., `grid[m - 1][n - 1]`). The robot can only move either down or right at any point in time.

Given the two integers m and n , return *the number of possible unique paths that the robot can take to reach the bottom-right corner*.

The test cases are generated so that the answer will be less than or equal to $2 * 10^9$.

Example 1:



Input: $m = 3, n = 7$
Output: 28

Example 2:

Input: $m = 3, n = 2$
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner.
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down

Constraints:

- $1 \leq m, n \leq 100$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def uniquePaths(self, m: int, n: int) -> int:
        # Initialize a 1D DP array with 1s
        dp = [1] * n

        # Update the DP array for each row
        for i in range(1, m):
            for j in range(1, n):
                dp[j] += dp[j-1]

        # Return the number of unique paths to the bottom-right corner
        return dp[n-1]
```

[50 Pow\(x, n\) \(link\)](#)

Description

Implement [pow\(x, n\)](#), which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Constraints:

- $-100.0 < x < 100.0$
- $-2^{31} \leq n \leq 2^{31}-1$
- n is an integer.
- Either x is not zero or $n > 0$.
- $-10^4 \leq x^n \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def myPow(self, x: float, n: int) -> float:
        if n < 0:
            x = 1 / x
            n = -n
        ans = 1.0
        while n:
            if n % 2:
                ans *= x
            x *= x
            n //= 2
        return ans
```

287 Find the Duplicate Number (link)

Description

Given an array of integers `nums` containing $n + 1$ integers where each integer is in the range $[1, n]$ inclusive.

There is only **one repeated number** in `nums`, return *this repeated number*.

You must solve the problem **without** modifying the array `nums` and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

Constraints:

- $1 \leq n \leq 10^5$
- `nums.length == n + 1`
- $1 \leq \text{nums}[i] \leq n$
- All the integers in `nums` appear only **once** except for **precisely one integer** which appears **two or more** times.

Follow up:

- How can we prove that at least one duplicate number must exist in `nums`?
- Can you solve the problem in linear runtime complexity?

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def findDuplicate(self, nums: List[int]) -> int:
        dup = set()
        for i in nums:
            if i in dup:
                return i
            dup.add(i)
```

74 Search a 2D Matrix (link)

Description

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if target is in matrix* or `false` *otherwise*.

You must write a solution in $O(\log(m * n))$ time complexity.

Example 1:

1	3	5	7
10	11	16	20
23	30	34	60

Input: `matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]]`, `target = 3`
Output: `true`

Example 2:

1	3	5	7
10	11	16	20
23	30	34	60

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false

Constraints:

- $m == \text{matrix.length}$
- $n == \text{matrix}[i].\text{length}$
- $1 \leq m, n \leq 100$
- $-10^4 \leq \text{matrix}[i][j], \text{target} \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def searchMatrix(self, matrix: List[List[int]], target: int) -> bool:
        if not matrix or not matrix[0]:
            return False

        m, n = len(matrix), len(matrix[0])
        start, end = 0, m * n - 1

        while start <= end:
            mid = (start + end) // 2
            element = matrix[mid // n][mid % n]
            if element == target:
                return True
            elif element < target:
                start = mid + 1
            else:
                end = mid - 1

        return False
```

[518 Coin Change II \(link\)](#)

Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the number of combinations that make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `0`.

You may assume that you have an infinite number of each kind of coin.

The answer is **guaranteed** to fit into a signed **32-bit** integer.

Example 1:

```
Input: amount = 5, coins = [1,2,5]
Output: 4
Explanation: there are four ways to make up the amount:
5=5
5=2+2+1
5=2+1+1+1
5=1+1+1+1+1
```

Example 2:

```
Input: amount = 3, coins = [2]
Output: 0
Explanation: the amount of 3 cannot be made up just with coins of 2.
```

Example 3:

```
Input: amount = 10, coins = [10]
Output: 1
```

Constraints:

- $1 \leq \text{coins.length} \leq 300$
- $1 \leq \text{coins}[i] \leq 5000$
- All the values of `coins` are **unique**.
- $0 \leq \text{amount} \leq 5000$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def change(self, amount: int, coins: List[int]) -> int:
        dp = [0] * (amount + 1)
        dp[0] = 1 # There is exactly one way to make amount 0 (by choosing no coins)

        for coin in coins:
            for i in range(coin, amount + 1):
                dp[i] += dp[i - coin]

        return dp[amount]
```

322 Coin Change [\(link\)](#)

Description

You are given an integer array `coins` representing coins of different denominations and an integer `amount` representing a total amount of money.

Return *the fewest number of coins that you need to make up that amount*. If that amount of money cannot be made up by any combination of the coins, return `-1`.

You may assume that you have an infinite number of each kind of coin.

Example 1:

Input: `coins = [1,2,5]`, `amount = 11`
Output: `3`
Explanation: `11 = 5 + 5 + 1`

Example 2:

Input: `coins = [2]`, `amount = 3`
Output: `-1`

Example 3:

Input: `coins = [1]`, `amount = 0`
Output: `0`

Constraints:

- $1 \leq \text{coins.length} \leq 12$
- $1 \leq \text{coins}[i] \leq 2^{31} - 1$
- $0 \leq \text{amount} \leq 10^4$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def coinChange(self, coins: List[int], amount: int) -> int:
        dp = [amount + 1] * (amount + 1)
        dp[0] = 0
        for coin in coins:
            for x in range(coin, amount + 1):
                dp[x] = min(dp[x], dp[x - coin] + 1)
        return dp[amount] if dp[amount] <= amount else -1
```

80 Remove Duplicates from Sorted Array II (link)

Description

Given an integer array `nums` sorted in **non-decreasing order**, remove some duplicates **in-place** such that each unique element appears **at most twice**. The **relative order** of the elements should be kept the **same**.

Since it is impossible to change the length of the array in some languages, you must instead have the result be placed in the **first part** of the array `nums`. More formally, if there are `k` elements after removing the duplicates, then the first `k` elements of `nums` should hold the final result. It does not matter what you leave beyond the first `k` elements.

Return `k` *after placing the final result in the first `k` slots of `nums`.*

Do **not** allocate extra space for another array. You must do this by **modifying the input array in-place** with $O(1)$ extra memory.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length

int k = removeDuplicates(nums); // Calls your implementation

assert k == expectedNums.length;
for (int i = 0; i < k; i++) {
    assert nums[i] == expectedNums[i];
}
```

If all assertions pass, then your solution will be **accepted**.

Example 1:

Input: `nums = [1,1,1,2,2,3]`

Output: `5, nums = [1,1,2,2,3,_,_]`

Explanation: Your function should return `k = 5`, with the first five elements of `nums` being `[1,1,2,2,3]`. It does not matter what you leave beyond the returned `k` (hence they are underscored).

Example 2:

Input: `nums = [0,0,1,1,1,1,2,3,3]`

Output: `7, nums = [0,0,1,1,2,3,3,_,_]`

Explanation: Your function should return `k = 7`, with the first seven elements of `nums` being `[0,0,1,1,2,3,3]`. It does not matter what you leave beyond the returned `k` (hence they are underscored).

Constraints:

- $1 \leq \text{nums.length} \leq 3 * 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def removeDuplicates(self, nums: List[int]) -> int:
        if len(nums) < 3:
            return len(nums)

        i = 1 # Pointer for the place to insert
        for j in range(2, len(nums)):
            if nums[j] != nums[i] or nums[j] != nums[i - 1]:
                i += 1
                nums[i] = nums[j]
        return i + 1 # i is index, so length is i+1
```

[344 Reverse String \(link\)](#)

Description

Write a function that reverses a string. The input string is given as an array of characters `s`.

You must do this by modifying the input array [in-place](#) with $O(1)$ extra memory.

Example 1:

```
Input: s = ["h","e","l","l","o"]  
Output: ["o","l","l","e","h"]
```

Example 2:

```
Input: s = ["H","a","n","n","a","h"]  
Output: ["h","a","n","n","a","H"]
```

Constraints:

- $1 \leq s.length \leq 10^5$
- `s[i]` is a [printable ascii character](#).

(scroll down for solution)

Solution

Language: *cpp*

Status: Accepted

```
class Solution {
public:
    void reverse(vector<char>& s, int l, int r){
        if(l >= r)
            return;
        else{
            swap(s[l],s[r]);
            reverse(s, l+1, r-1);
        }
    }

    void reverseString(vector<char>& s) {
        reverse(s,0,s.size()-1);
    }
};
```

[198 House Robber \(link\)](#)

Description

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and **it will automatically contact the police if two adjacent houses were broken into on the same night**.

Given an integer array `nums` representing the amount of money of each house, return *the maximum amount of money you can rob tonight **without alerting the police***.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 4

Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.

Example 2:

Input: `nums = [2,7,9,3,1]`

Output: 12

Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.

Constraints:

- $1 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 400$

(scroll down for solution)

Solution

Language: *cpp*

Status: Accepted

```
class Solution {
public:
    int rob(vector<int>& nums) {
        int n=nums.size();
        if(n>1)
            nums[n-2]=max(nums[n-1],nums[n-2]);
        for(int i=n-3;i>=0;i--)
        {
            nums[i]=max(nums[i]+nums[i+2],nums[i+1]);
        }
        return nums[0];
    }
};
```

[1250 Longest Common Subsequence \(link\)](#)

Description

Given two strings `text1` and `text2`, return *the length of their longest **common subsequence***. If there is no **common subsequence**, return `0`.

A **subsequence** of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- For example, "ace" is a subsequence of "abcde".

A **common subsequence** of two strings is a subsequence that is common to both strings.

Example 1:

Input: `text1 = "abcde", text2 = "ace"`

Output: `3`

Explanation: The longest common subsequence is "ace" and its length is 3.

Example 2:

Input: `text1 = "abc", text2 = "abc"`

Output: `3`

Explanation: The longest common subsequence is "abc" and its length is 3.

Example 3:

Input: `text1 = "abc", text2 = "def"`

Output: `0`

Explanation: There is no such common subsequence, so the result is 0.

Constraints:

- `1 <= text1.length, text2.length <= 1000`
- `text1` and `text2` consist of only lowercase English characters.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def longestCommonSubsequence(self, text1: str, text2: str) -> int:
        m = len(text1)
        n = len(text2)
        dp = []

        for i in range(m+1):
            row = []
            for j in range(n+1):
                row.append(-1)
            dp.append(row)
        print(dp)

        for i in range(m+1):
            for j in range(n+1):
                # print(i)
                # print(j)
                if (i == 0 or j==0):
                    dp[i][j] = 0
                elif(text1[i -1] == text2[j-1]):
                    dp[i][j] = 1+ dp[i-1][j-1]
                else:
                    dp[i][j] = max(dp[i-1][j],dp[i][j - 1])
        return dp[-1][-1]
```

[70 Climbing Stairs \(link\)](#)

Description

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

Input: $n = 2$

Output: 2

Explanation: There are two ways to climb to the top.

1. 1 step + 1 step
2. 2 steps

Example 2:

Input: $n = 3$

Output: 3

Explanation: There are three ways to climb to the top.

1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step

Constraints:

- $1 \leq n \leq 45$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def climbStairs(self, n: int) -> int:
        if(n ==1):
            return 1
        single_step = 1
        double_step = 2
        permutations = 0
        for i in range(3, n + 1):
            permutations = single_step + double_step
            single_step = double_step
            double_step = permutations
        return double_step
```

[153 Find Minimum in Rotated Sorted Array \(link\)](#)

Description

Suppose an array of length n sorted in ascending order is **rotated** between 1 and n times. For example, the array `nums = [0,1,2,4,5,6,7]` might become:

- `[4,5,6,7,0,1,2]` if it was rotated 4 times.
- `[0,1,2,4,5,6,7]` if it was rotated 7 times.

Notice that **rotating** an array `[a[0], a[1], a[2], ..., a[n-1]]` 1 time results in the array `[a[n-1], a[0], a[1], a[2], ..., a[n-2]]`.

Given the sorted rotated array `nums` of **unique** elements, return *the minimum element of this array*.

You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [3,4,5,1,2]`

Output: 1

Explanation: The original array was `[1,2,3,4,5]` rotated 3 times.

Example 2:

Input: `nums = [4,5,6,7,0,1,2]`

Output: 0

Explanation: The original array was `[0,1,2,4,5,6,7]` and it was rotated 4 times.

Example 3:

Input: `nums = [11,13,15,17]`

Output: 11

Explanation: The original array was `[11,13,15,17]` and it was rotated 4 times.

Constraints:

- $n == \text{nums.length}$
- $1 \leq n \leq 5000$
- $-5000 \leq \text{nums}[i] \leq 5000$
- All the integers of `nums` are **unique**.
- `nums` is sorted and rotated between 1 and n times.

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution(object):
    def findMin(self, nums):
        """
        :type nums: List[int]
        :rtype: int
        """
        # If the list has just one element then return that element.
        if len(nums) == 1:
            return nums[0]

        # left pointer
        left = 0
        # right pointer
        right = len(nums) - 1

        # if the last element is greater than the first element then there is no
        # e.g. 1 < 2 < 3 < 4 < 5 < 7. Already sorted array.
        # Hence the smallest element is first element. A[0]
        if nums[right] > nums[0]:
            return nums[0]

        # Binary search way
        while right >= left:
            # Find the mid element
            mid = left + (right - left) // 2
            # if the mid element is greater than its next element then mid+1 element
            # This point would be the point of change. From higher to lower value
            if nums[mid] > nums[mid + 1]:
                return nums[mid + 1]
            # if the mid element is lesser than its previous element then mid element
            if nums[mid - 1] > nums[mid]:
                return nums[mid]

            # if the mid elements value is greater than the 0th element this means
            # the least value is still somewhere to the right as we are still dealing with
            if nums[mid] > nums[0]:
                left = mid + 1
            # if nums[0] is greater than the mid value then this means the smallest element
            else:
                right = mid - 1
```

[238 Product of Array Except Self \(link\)](#)

Description

Given an integer array `nums`, return *an array* `answer` *such that* `answer[i]` *is equal to the product of all the elements of* `nums` *except* `nums[i]`.

The product of any prefix or suffix of `nums` is **guaranteed** to fit in a **32-bit** integer.

You must write an algorithm that runs in $O(n)$ time and without using the division operation.

Example 1:

Input: `nums = [1,2,3,4]`
Output: `[24,12,8,6]`

Example 2:

Input: `nums = [-1,1,0,-3,3]`
Output: `[0,0,9,0,0]`

Constraints:

- $2 \leq \text{nums.length} \leq 10^5$
- $-30 \leq \text{nums}[i] \leq 30$
- The input is generated such that `answer[i]` is **guaranteed** to fit in a **32-bit** integer.

Follow up: Can you solve the problem in $O(1)$ extra space complexity? (The output array **does not** count as extra space for space complexity analysis.)

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def productExceptSelf(self, nums: List[int]) -> List[int]:
        output = [1]
        for i in range(len(nums)-1):
            output.append(output[-1] * nums[i])

        # print(output)

        temp = 1
        for i in range(len(nums)-1,-1,-1):
            # print("Output")
            # print(output)
            output[i] *= temp
            temp *= nums[i]
        #     print("Temp")
        #     print(temp)

        return output
```

[217 Contains Duplicate \(link\)](#)

Description

Given an integer array `nums`, return `true` if any value appears **at least twice** in the array, and return `false` if every element is distinct.

Example 1:

Input: `nums = [1,2,3,1]`

Output: `true`

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: `nums = [1,2,3,4]`

Output: `false`

Explanation:

All elements are distinct.

Example 3:

Input: `nums = [1,1,1,3,3,4,3,2,4,2]`

Output: `true`

Constraints:

- $1 \leq \text{nums.length} \leq 10^5$
- $-10^9 \leq \text{nums}[i] \leq 10^9$

(scroll down for solution)

Solution

Language: python3

Status: Accepted

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        set1 = set(nums)
        if len(nums) == len(set1):
            return False
        else:
            return True
```

